



電子遊戲銷售

by 第四組

112029038 林資竣

112029039 黃柏誠

112029023 游雲翊

112029051 黃柏堯

112012067 秦晨恩

背景

組員未來有志於電子遊戲開發，希望了解一款遊戲是否能成為暢銷作品（Hit），主要受到哪些因素影響，本研究使用全球電子遊戲銷售資料，分析遊戲的平台、類型、發行商等特徵，透過資料分析與視覺化，探討不同特徵對「成為暢銷遊戲」的影響程度，研究結果可作為遊戲開發與行銷策略的參考依據

研究目的

建立「暢銷遊戲」定義 ($\text{Global_Sales} > \text{中位數} = 0.17 \text{ 百萬套}$)

找出影響遊戲是否成為暢銷作品的關鍵特徵

建立分類預測模型，預測遊戲是否可能成為暢銷遊戲

根據分析與模型結果提出遊戲開發與行銷策略建議

資料描述

各種遊戲銷售紀錄 總共16598筆資料 11個蒐集項目

	A	B	C	D	E	F	G	H	I	J	K
1	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
2		1 Wii Sports	Wii		2006 Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
3		2 Super Mario Bros.	NES		1985 Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
4		3 Mario Kart Wii	Wii		2008 Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
5		4 Wii Sports Resort	Wii		2009 Sports	Nintendo	15.75	11.01	3.28	2.96	33
6		5 Pokemon Red/Pokemon Blue	GB		1996 Role-Playing	Nintendo	11.27	8.89	10.22	1	31.37
7		6 Tetris	GB		1989 Puzzle	Nintendo	23.2	2.26	4.22	0.58	30.26
8		7 New Super Mario Bros. Wii	DS		2006 Platform	Nintendo	11.38	9.23	6.5	2.9	30.01
9		8 Wii Play	Wii		2006 Misc	Nintendo	14.03	9.2	2.93	2.85	29.02
10		9 New Super Mario Bros. Wii	Wii		2009 Platform	Nintendo	14.59	7.06	4.7	2.26	28.62
11		10 Duck Hunt	NES		1984 Shooter	Nintendo	26.93	0.63	0.28	0.47	28.31
12		11 Nintendogs	DS		2005 Simulation	Nintendo	9.07	11	1.93	2.75	24.76
13		12 Mario Kart DS	DS		2005 Racing	Nintendo	9.81	7.57	4.13	1.92	23.42
14		13 Pokemon Gold/Pokemon Silver	GB		1999 Role-Playing	Nintendo	9	6.18	7.2	0.71	23.1
15		14 Wii Fit	Wii		2007 Sports	Nintendo	8.94	8.03	3.6	2.15	22.72
16		15 Wii Fit Plus	Wii		2009 Sports	Nintendo	9.09	8.59	2.53	1.79	22
17		16 Kinect Adventures!	X360		2010 Misc	Microsoft Game Studios	14.97	4.94	0.24	1.67	21.82
18		17 Grand Theft Auto V	PS3		2013 Action	Take-Two Interactive	7.01	9.27	0.97	4.14	21.4
19		18 Grand Theft Auto: San Andreas	PS2		2004 Action	Take-Two Interactive	9.43	0.4	0.41	10.57	20.81

研究所使用的資料集為公開的 Video Game Sales Dataset,

其資料來源為 VGChartz 所彙整之全球遊戲銷售紀錄。

資料內容涵蓋 1980 至 2020年間發售的遊戲，包括遊戲基本
資訊與各地區銷售量。

資料描述

	A	B	C	D	E	F	G	H	I	J	K
1	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
2		1 Wii Sports	Wii	2006	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
3		2 Super Mario Bros.	NES	1985	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
4		3 Mario Kart Wii	Wii	2008	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
5		4 Wii Sports Resort	Wii	2009	Sports	Nintendo	15.75	11.01	3.28	2.96	33
6		5 Pokemon Red/Pokemon Blue	GB	1996	Role-Playing	Nintendo	11.27	8.89	10.22	1	31.37
7		6 Tetris	GB	1989	Puzzle	Nintendo	23.2	2.26	4.22	0.58	30.26
8		7 New Super Mario Bros.	DS	2006	Platform	Nintendo	11.38	9.23	6.5	2.9	30.01
9		8 Wii Play	Wii	2006	Misc	Nintendo	14.03	9.2	2.93	2.85	29.02
10		9 New Super Mario Bros.	Wii	2009	Platform	Nintendo	14.59	7.06	4.7	2.26	28.62
11		10 Duck Hunt	NES	1984	Shooter	Nintendo	26.93	0.63	0.28	0.47	28.31
12		11 Nintendogs	DS	2005	Simulation	Nintendo	9.07	11	1.93	2.75	24.76
13		12 Mario Kart DS	DS	2005	Racing	Nintendo	9.81	7.57	4.13	1.92	23.42
14		13 Pokemon Gold/Pokemon Silver	GB	1999	Role-Playing	Nintendo	9	6.18	7.2	0.71	23.1
15		14 Wii Fit	Wii	2007	Sports	Nintendo	8.94	8.03	3.6	2.15	22.72
16		15 Wii Fit Plus	Wii	2009	Sports	Nintendo	9.09	8.59	2.53	1.79	22
17		16 Kinect Adventures!	X360	2010	Misc	Microsoft Game Studios	14.97	4.94	0.24	1.67	21.82
18		17 Grand Theft Auto V	PS3	2013	Action	Take-Two Interactive	7.01	9.27	0.97	4.14	21.4
19		18 Grand Theft Auto: San Andreas	PS2	2004	Action	Take-Two Interactive	9.43	0.4	0.41	10.57	20.81

蒐集項目含遊戲全球銷售**排名**、遊戲**名稱**、遊戲**平台**、發售**年份**、遊戲**類型**、**發行商**、**北美銷售量**、**歐洲銷售量**、**日本銷售量**、**其他地區銷售量**、**全球銷售總合**

欄位介紹

1	Rank	遊戲全球銷售排名	連續型資料
2	Name	遊戲名稱	類別型資料
3	Platform	遊戲平台	類別型資料
4	Year	發售年份	連續型資料
5	Genre	遊戲類型	類別型資料

欄位介紹

6	Publisher	發行商	類別型資料
7	NA_Sales	北美銷售（百萬套）	連續型資料
8	EU_Sales	歐洲銷售（百萬套）	連續型資料
9	JP_Sales	日本銷售（百萬套）	連續型資料
10	Other_Sales	其他地區銷售（百萬套）	連續型資料
11	Global_Sales	全球銷售總和（百萬套）	連續型資料

資料型態

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16598 entries, 0 to 16597
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Rank                   16598 non-null  int64
1   Name                   16598 non-null  object
2   Platform               16598 non-null  object
3   Year                   16327 non-null  object
4   Genre                  16598 non-null  object
5   Publisher              16540 non-null  object
6   NA_Sales                16598 non-null  float64
7   EU_Sales                16598 non-null  float64
8   JP_Sales                16598 non-null  float64
9   Other_Sales            16598 non-null  float64
10  Global_Sales           16596 non-null  float64
dtypes: float64(5), int64(1), object(5)
memory usage: 1.4+ MB
None
```

連續型資料:

Rank
NA_Sales
EU_Sales
JP_Sales
Other_Sales
Global_Sales

類別型資料:

Year
Name
Platform
Genre
Publisher

資料前處理

檢查遺漏值



```
df.isnull().sum()
```

Platform	0
Year	271
Genre	0
Publisher	58
NA_Sales	0
EU_Sales	0
JP_Sales	0
Other_Sales	0
Global_Sales	2
dtype:	int64

可以發現Year 欄位缺失 271 筆

Publisher 缺失 58 筆

Global_Sales 缺失 2 筆

其他欄位皆無缺失

接著依照變數性質進行適當的缺失值處理

檢查遺漏值

```
import pandas as pd

# 讀取資料
df = pd.read_csv("vgsales.csv")

# 1. 刪除不需要的欄位 (Rank、Name)
df = df.drop(columns=["Rank", "Name"])

# 2. 缺失值處理
# Global_Sales: 重要欄位 → 刪除缺失值
df = df.dropna(subset=["Global_Sales"])

# Publisher: 類別欄位 → 補為 Unknown
df["Publisher"] = df["Publisher"].fillna("Unknown")

# Year: 年份不可補 → 刪除缺失
df = df.dropna(subset=["Year"])

# 3. 確認缺失值是否都處理完畢
print(df.isnull().sum())

# 查看處理後的筆數
print("處理後筆數:", len(df))
```

Platform 0
Year 0
Genre 0
Publisher 0
NA_Sales 0
EU_Sales 0
JP_Sales 0
Other_Sales 0
Global_Sales 0
dtype: int64
處理後筆數: 16325

遺漏值檢查結果

Year 缺失 271 筆 (刪除)

Publisher 缺失 58 筆 (補 Unknown)

Global_Sales 缺失 2 筆 (刪除)

其他欄位皆完整

處理後無缺失值，剩餘 16,325 筆資料。

特徵剔除 (FEATURE REMOVAL)

Rank 為依照 **global_sales** 排序所得，與**目標變數高度相關**。

若納入模型會造成資料洩漏，因此在建模前予以移除。

Name 純文字資料（非結構化），無法直接用於機器學習

字串種類太多（16000+）→ 會造成維度爆炸

與是否暢銷**無明確因果關係**

因此刪除避免模型混亂

```
import pandas as pd

# 讀取資料 (csv 與 notebook 在同一資料夾)
df = pd.read_csv("vgsales.csv")

# 刪除 Rank 欄位
df = df.drop(columns=["Rank", "Name"])

# 查看前幾筆資料
df.head()
```



特徵剔除

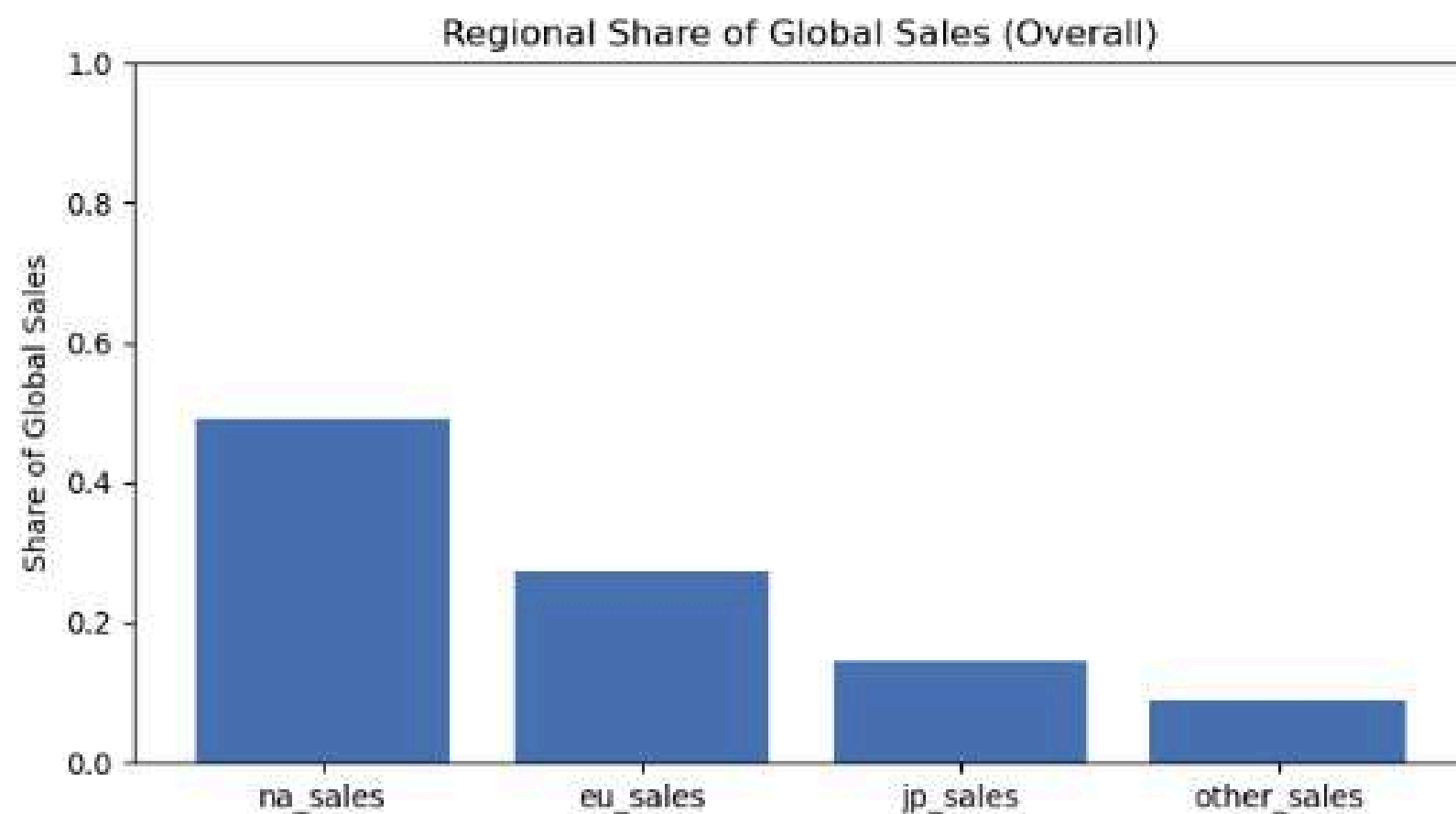
(FEATURE REMOVAL)

	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	Wii	2006	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
1	NES	1985	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
2	Wii	2008	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
3	Wii	2009	Sports	Nintendo	15.75	11.01	3.28	2.96	33.00
4	GB	1996	Role-Playing	Nintendo	11.27	8.89	10.22	1.00	31.37



刪除不必要資料

```
=== 各地區銷售量佔全球比例 (整體) ===  
na_sales    49.13%  
eu_sales    27.31%  
jp_sales    14.56%  
other_sales   8.95%  
dtype: object
```



全球銷售結構高度偏向北美市場，顯示地區銷售量具有主導性與不平衡問題。

- 因此決定刪除地區銷售套數的資料

鎖定研究範圍(平台)

本研究聚焦於 同世代(第七世代)主流家用

主機平台：

Wii、PS3、Xbox 360

這三個平台：

發行時間相近 (約 2006–2013)

市場定位與硬體世代相同

可避免跨世代平台造成的銷售規模偏差

PC 納入但限制同世代年份

- PC 平台橫跨多個世代，若不加限制，會：
 - 引入早期與近代遊戲
 - 造成時間因素與平台效果混淆
- 因此本研究：
 - 以 Wii / PS3 / Xbox 360 在資料中出現的最早與最晚年份
 - 定義「同世代時間範圍」
 - 僅保留該期間內的 PC 遊戲

```
# 1) 平台固定：WII / PS3 / X360 / PC
# =====
keep_platforms = ["WII", "PS3", "X360", "PC"]
df_p = df[df["platform"].isin(keep_platforms)].copy()

print("\n只留指定平台後 shape:", df_p.shape)
print("平台分布:\n", df_p["platform"].value_counts())

if df_p.empty:
    raise ValueError("篩選後資料為空：請檢查 df['platform'].unique() 的平台寫法是否不同。")

# =====
# 2) PC 同世代年份限制
# 同世代年份 = Wii/PS3/X360 在資料中出現的 min~max year
# =====
base_platforms = ["WII", "PS3", "X360"]
base = df_p[df_p["platform"].isin(base_platforms)].copy()

min_year = int(base["year"].min())
max_year = int(base["year"].max())

print(f"\n同世代年份範圍 (由 Wii/PS3/X360 決定) : {min_year} ~ {max_year}")

df_p = df_p[df_p["year"].between(min_year, max_year)].copy()

print("套用同世代年份後 shape:", df_p.shape)
print("平台分布 (含PC同世代限制後):\n", df_p["platform"].value_counts())

if df_p.empty:
    raise ValueError("套用同世代年份後資料為空：代表該年份範圍內資料不足。")
```

鎖定研究範圍(遊戲類型)

原始資料包含十多種類型，部分
類型樣本數極少

為避免：

維度過高、模型不穩定

本研究：

依樣本數選出

前五大遊戲類型

其餘類型統一歸為 Others

```
# 5) 類型：Top5 + Others (不刪資料)
# =====
top5_genres = df_p["genre"].value_counts().head(5).index.tolist()
df_p["genre_group"] = df_p["genre"].where(df_p["genre"].isin(top5_genres), "Others")

print("\nTop5 Genres:", top5_genres)
print("genre_group 分布:\n", df_p["genre_group"].value_counts())
```

建立HIT

為將「遊戲銷售表現」轉換為可供模型學習的分類問題，
本研究將連續型的 Global_Sales（全球銷售量）

轉換為二元目標變數 HIT：

- HIT = 1：銷售量 $\geq$ 全體樣本之中位數（暢銷遊戲）
- HIT = 0：銷售量 $<$ 中位數（非暢銷遊戲）

採用 中位數作為切分門檻，可降低極端暢銷作品

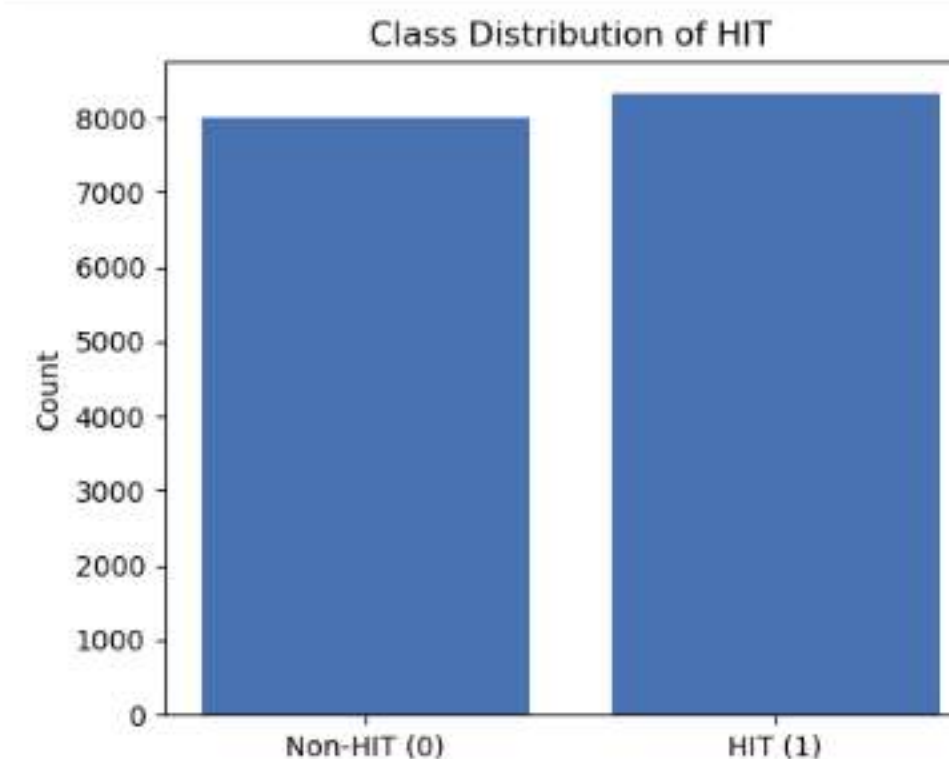
對平均值的影響，並使資料分布更為平衡，有助於提升分類模型的穩定性與可解釋性。

```
print("\n=== HIT 分布 (數量) ===")
print(df_model["hit"].value_counts().rename({0:"Non-HIT", 1:"HIT"}))

print("\n=== HIT 分布 (比例) ===")
print((df_model["hit"].value_counts(normalize=True)
        .rename({0:"Non-HIT", 1:"HIT"}) * 100).round(2).astype(str) + "%")
```

```
=== HIT 分布 (數量) ===
HIT      8327
Non-HIT   7998
Name: hit, dtype: int64
```

```
=== HIT 分布 (比例) ===
HIT      51.01%
Non-HIT   48.99%
Name: hit, dtype: object
```



資料編碼

將「發行商 (Publisher) 進行編碼」，
我們選擇計算每個 Publisher 成功遊戲數量
排序 → 取前 10 名

代表的是：最容易出「暢銷作品」的發行商
執行完後會得到：

✓ **top10_publishers**

→ 10 個最會出暢銷遊戲的發行商 (例如
Nintendo、EA、Activision 等)

✓ Top10Publisher 欄位 (0 / 1)

- 1 = 此發行商是前十名暢銷大品牌
- 0 = 不是大品牌

```
# =====  
# 4) Top10 Publisher (二元特徵)  
# 用 hit 的總數挑 Top10 (比較符合「暢銷」概念)  
# =====  
top_publishers = (  
    df_p.groupby("publisher")["hit"].sum()  
    .sort_values(ascending=False)  
    .head(10)  
    .index  
)  
df_p["top10_publisher"] = df_p["publisher"].isin(top_publishers).astype(int)  
print("\nTop10 Publisher:", list(top_publishers))
```

資料編碼

平台 (Platform)、遊戲類型 (Genre) 屬於類別型資料
機器學習模型無法直接處理文字類別
需將類別資料轉換為數值形式，才能進行模型訓練

- 採用 One-Hot Encoding
- 將每一個平台與遊戲類型轉換為獨立的 0 / 1 特徵欄位
- 使模型能分別學習各類別對「是否成為暢銷遊戲 (Hit)」的影響

```
# =====  
# 10) One-Hot Encoding + 輸出 CSV (給老師講解/丟模型用)  
# 不用 drop_first: 比較好講解 (不會少一個類別)  
# =====  
X = df_final[["platform", "genre_group", "top10_publisher"]].copy()  
y = df_final["hit"].copy()  
  
X_encoded = pd.get_dummies(X, columns=["platform", "genre_group"], drop_first=False)  
  
df_onehot = X_encoded.copy()  
df_onehot["hit"] = y.values  
  
# 輸出檔案  
df_final.to_csv("vgsales_focus_final.csv", index=False) # 未編碼、可解釋  
df_onehot.to_csv("vgsales_focus_onehot.csv", index=False) # One-Hot 後、可丟模型
```

資料編碼

年份不作為模型輸入特徵

年份僅用於：

限定研究範圍（控制同世代主機）

不納入模型原因：

年份容易反映產業成熟度與市場規模變化

可能掩蓋平台與類型本身對暢銷與否的影響

為提升模型可解釋性與穩定性，因此不納入模型訓練

```
# =====  
# 6) 最終資料 (不要 year / 不要 era)  
# =====  
df_final = df_p[["platform", "genre_group", "top10_publisher", "hit"]].copy()  
  
print("\n最終資料 shape:", df_final.shape)  
print("最終欄位:", df_final.columns.tolist())
```

視覺化分析

平台 × HIT RATE (長條圖)

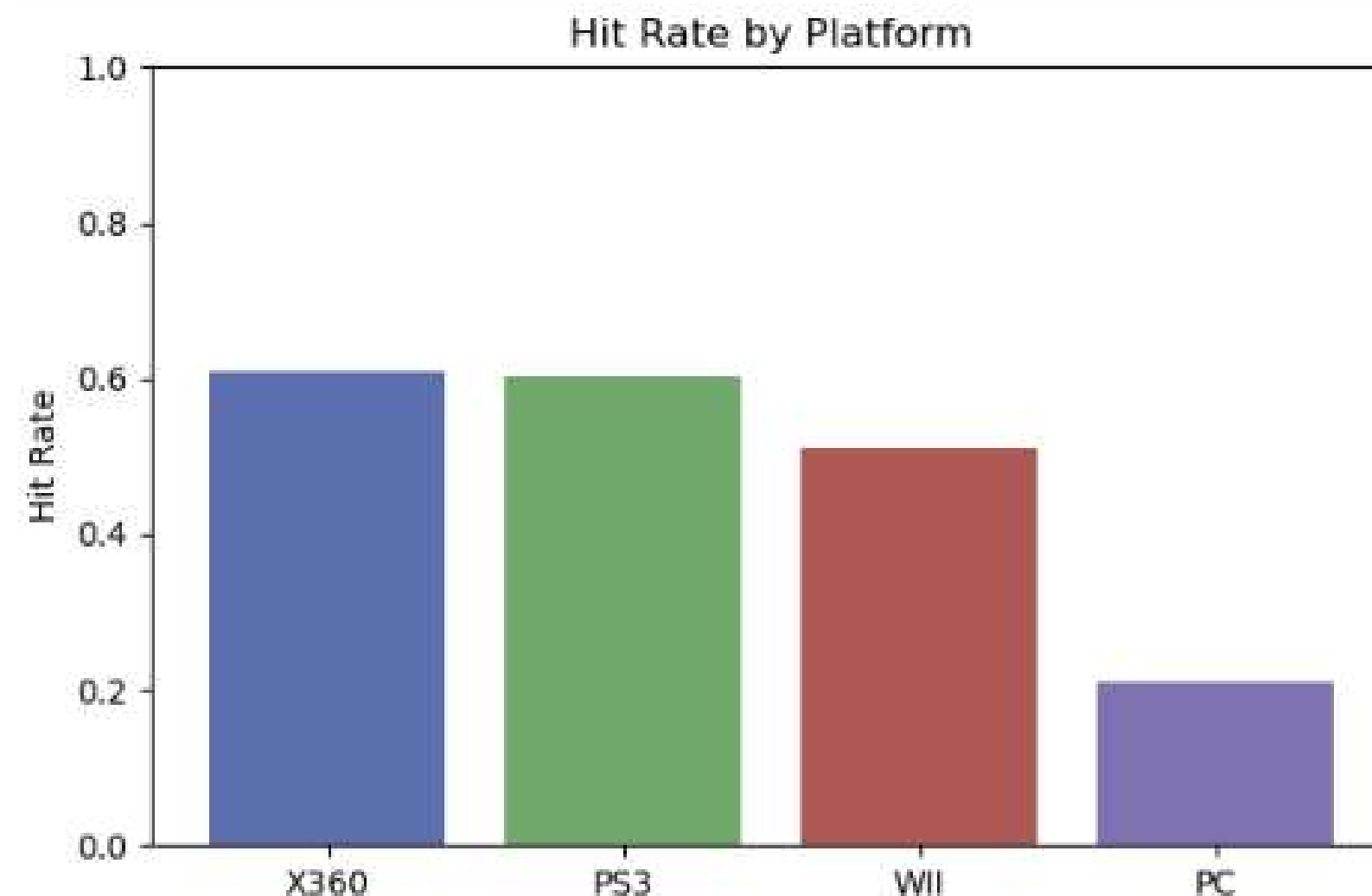
從結果可觀察到，主機平台（Xbox 360、PS3、Wii）之間的 Hit Rate 數值相對接近，

其中 Xbox 360 與 PS3 略高，約落在 60% 左右，

Wii 則略低，而 PC 平台的 Hit Rate 明顯偏低，

可能與 PC 市場銷售分散有關。

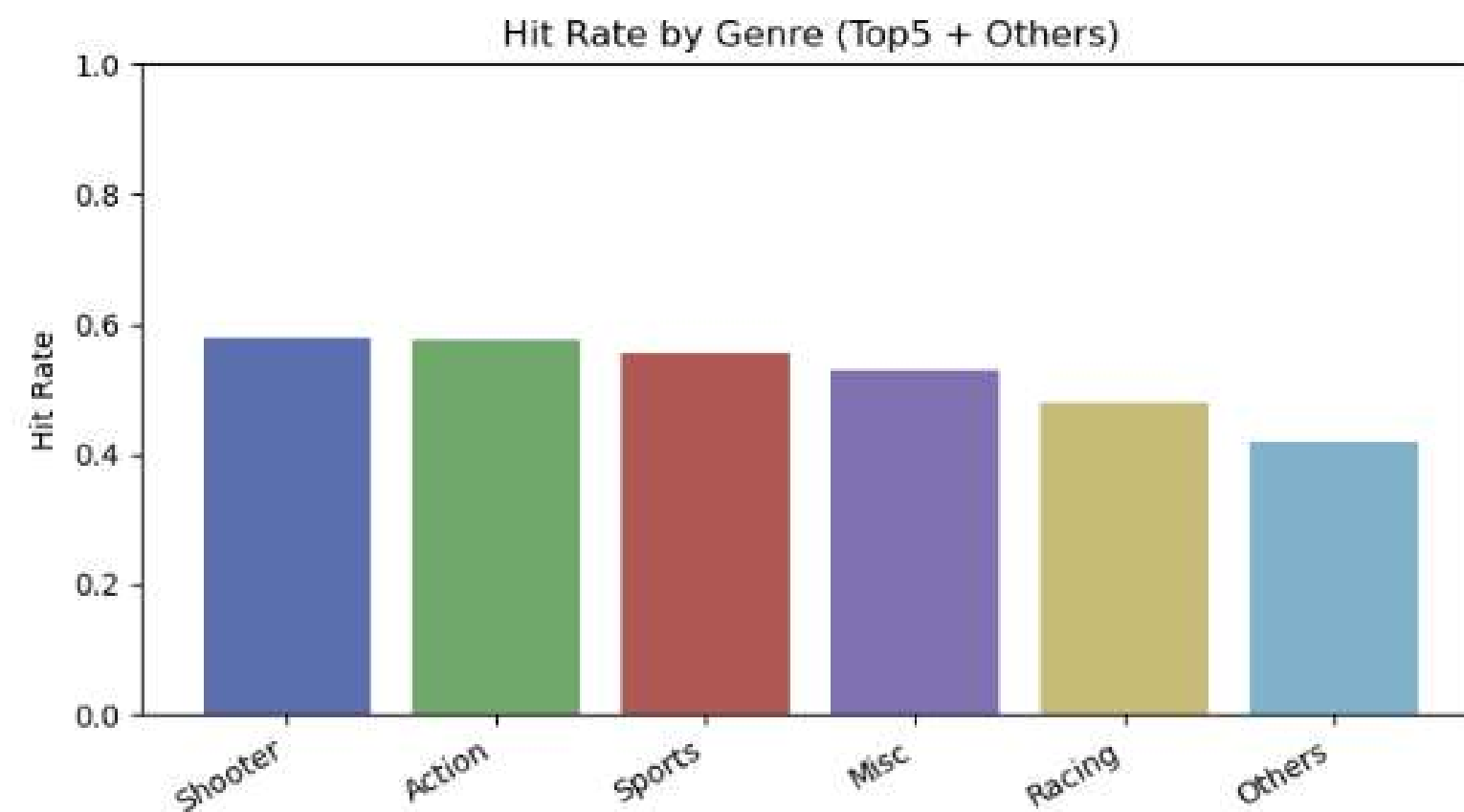
此結果顯示「平台選擇」是影響遊戲是否成為暢銷作品的重要因素之一。



類型 × HIT RATE (長條圖)

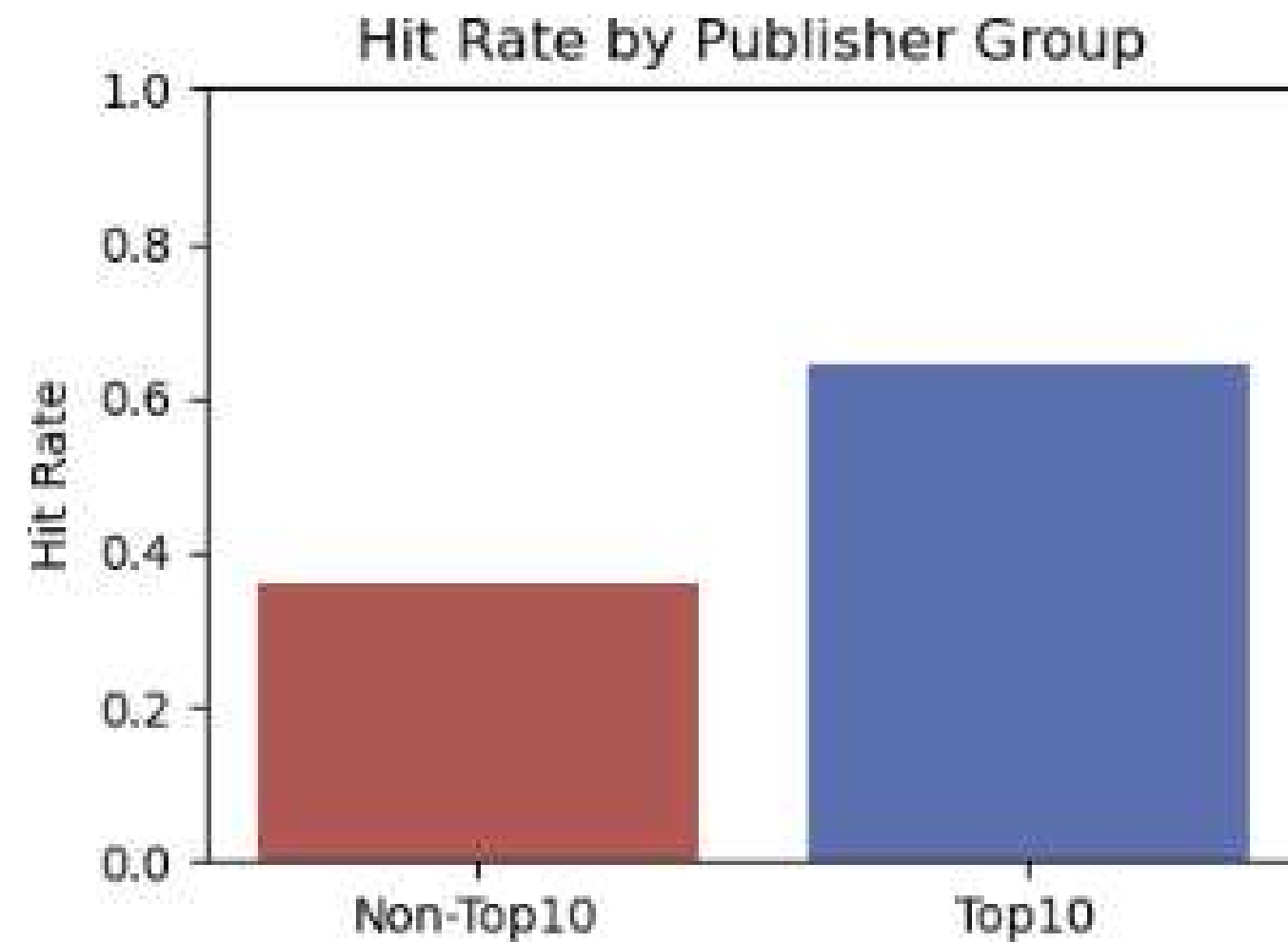
Shooter (射擊)、Action (動作)、Sports (運動) 與 Misc (綜合) 這四種類型的暢銷率位居前列，約在 53% 至 58% 之間。相較之下，Racing (競速) 與 Others (其他) 類型的暢銷率較低。

雖然各類型間的 Hit Rate 差距有限，但結果顯示遊戲類型對成功機率具有方向性影響，並可作為分類模型中的有效特徵之一。



發行商影響 (TOP10 VS 非TOP10)

結果顯示，Top10 發行商所推出的遊戲，其 Hit Rate 明顯高於非 Top10 發行商。
Top10 發行商的暢銷比例約為 64%，
而非 Top10 發行商僅約 36%。
此差異可能來自於大型發行商在品牌知名度、行銷資源與通路優勢上的影響，
顯示發行商背景是影響銷售成功的重要因素之一。



實驗設計

實驗設計

本研究將「是否為暢銷遊戲」視為二元分類問題：

目標變數 (Y)：HIT，1=暢銷 ($\text{GLOBAL_SALES} > 0.17$)，0=非暢銷

解釋變數 (X) 包含：

遊戲平台 (PLATFORM)

遊戲類型 (GENRE GROUP)

發行商相關特徵 (例如 TOP10PUBLISHER)

研究目標： 建立分類模型，預測遊戲是否可能成為暢銷作品，並分析各特徵的重要性。

資料切分

為了建立能泛化至未來新遊戲的預測模型，本研究將資料分為：

- 訓練集 (TRAINING SET)：70%
- 供模型學習特徵與銷售表現之間的關係。
- 測試集 (TESTING SET)：30%
- 用於評估模型在未見資料上的預測能力。

本研究以全球銷售量的中位數作為暢銷遊戲 (HIT=1) 的判定標準，使兩類別在整體資料中比例大致相當。

為確保訓練集與測試集中 HIT 類別比例與原始資料一致，避免隨機切分造成類別比例偏移，

因此在資料切分時採用分層抽樣 (STRATIFIED SAMPLING) 方式。

資料切分

特徵欄位 (X) 包含：

- 平台 (PLATFORM)
- 遊戲類型 (GENRE GROUP)
- 是否為前十大發行商 (TOP10PUBLISHER)

目標欄位 (Y)：

- HIT (0/1) 是否為暢銷遊戲
- 1：遊戲全球銷售量高於資料集中位數 (暢銷)
0：遊戲全球銷售量低於資料集中位數 (非暢銷)

```
from sklearn.model_selection import train_test_split

# =====
# 7:3 切分訓練集 / 測試集
# =====
X = df_onehot.drop(columns=["hit"]).copy()
y = df_onehot["hit"].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,      # ✓ 30% 當測試集 → 7:3
    random_state=42,
    stratify=y          # ✓ 保持 0/1 比例一致 (分類很重要)
)

print("X_train:", X_train.shape, "X_test:", X_test.shape)
print("y_train hit rate:", y_train.mean(), "y_test hit rate:", y_test.mean())
```

```
X_train: (3238, 11) X_test: (1388, 11)
y_train hit rate: 0.5111179740580605 y_test hit rate: 0.5115273775216138
```

資料切分

特徵欄位 (X) :

- 平台 (PLATFORM)

類別型變數

以 ONE-HOT ENCODING 方式轉換為:

PLATFORM_WII

PLATFORM_PS3

PLATFORM_X360

PLATFORM_PC

- 是否為前十大發行商 (TOP10PUBLISHER)

二元變數

1: 前十大發行商

0: 非前十大發行商

- 遊戲類型 (GENRE GROUP)

類別型變數

以 ONE-HOT ENCODING 方式轉換為:

GENRE_GROUP_ACTION

GENRE_GROUP_SPORTS

GENRE_GROUP_ROLE-PLAYING

GENRE_GROUP_SHOOTER

GENRE_GROUP_MISC

GENRE_GROUP_OTHERS

預期成果

不同的平台、遊戲類型，以及是否由大型發行商發行，皆可能對遊戲是否成為暢銷作品（Hit）產生不同程度的影響。本研究透過資料分析與分類模型，探討這些特徵與遊戲暢銷表現之間的關聯性，以了解哪些因素能提升遊戲成為暢銷作品的機率。透過分析同世代平台（Wii、PS3、X360 與 PC）之遊戲資料，本研究期望能提供遊戲開發商與發行商在平台選擇、遊戲類型規劃，以及發行策略上的參考依據，進而提升產品成功機率，並降低新作品在市場上表現不如預期的風險。

評估指標

Accuracy(準確率):

- 整體預測正確的比例
- 資料不平衡時會有影響
- $(TP+TN)/(TP+TN+FP+FN)$

Precision(精確率):

- 模型「說有」的樣本中，有多少是真的有
- $TP/(TP+FP)$
- 越高代表模型預測為正的結果越可

靠

RECALL(召回率):

- 實際真的有的樣本中，有多少被模型抓到
- $TP/(TP+FN)$
- 越高代表模型能捕捉到越多實際為正的樣本

實際\預測	預測為正(1)	預測為負(0)
實際為正(1)	TP(真正)	FN(漏判)
實際為負(0)	FP(誤判)	TN(真負)

評估指標

F1-Score(準確率):

- 同時兼顧精確率與召回率的一個指標
- $2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$

ROC Curve:

- 描述模型在不同判斷閾值下的表現
- x軸:FPR: $FP / (FP + TN)$
- y軸:TPR: $TP / (TP + FN)$
- 曲線越靠左上角 → 模型越好

AUC(Area Under Curve):

- ROC 曲線下面積
- 衡量模型對結果的區辨能力，數值越大代表模型效能越佳

實際\預測	預測為正(1)	預測為負(0)
實際為正(1)	TP(真正)	FN(漏判)
實際為負(0)	FP(誤判)	TN(真負)

決策樹

超參數調整

```
param_grid = {  
    "max_depth": [2, 3, 4, 5, 6, 8, 10, None],  
    "min_samples_split": [2, 5, 10, 20],  
    "min_samples_leaf": [1, 2, 5, 10],  
    "criterion": ["gini", "entropy"]  
}
```

```
cv = StratifiedKFold(n_splits=N_SPLITS, shuffle=True,  
    random_state=RANDOM_STATE)  
base_model = DecisionTreeClassifier(  
    random_state=RANDOM_STATE,  
    class_weight="balanced"  
)
```

```
grid = GridSearchCV(  
    estimator=base_model,  
    param_grid=param_grid,  
    scoring="roc_auc",  
    cv=cv,  
    n_jobs=-1 )
```

```
grid.fit(X_train, y_train)  
best_params = grid.best_params_  
best_model = grid.best_estimator_
```

確保每一折的訓練集裡面0/1比例平衡

超參數調整

```
print("Best params:", best_params)
print("Best CV AUC:", grid.best_score_)
```

```
=== Converged (Selected) Hyperparameters ===
Best params: {'criterion': 'gini', 'max_depth': 5, 'min_samples_leaf': 5, 'min_samples_split': 2}
Best CV AUC: 0.7143583881645021
```



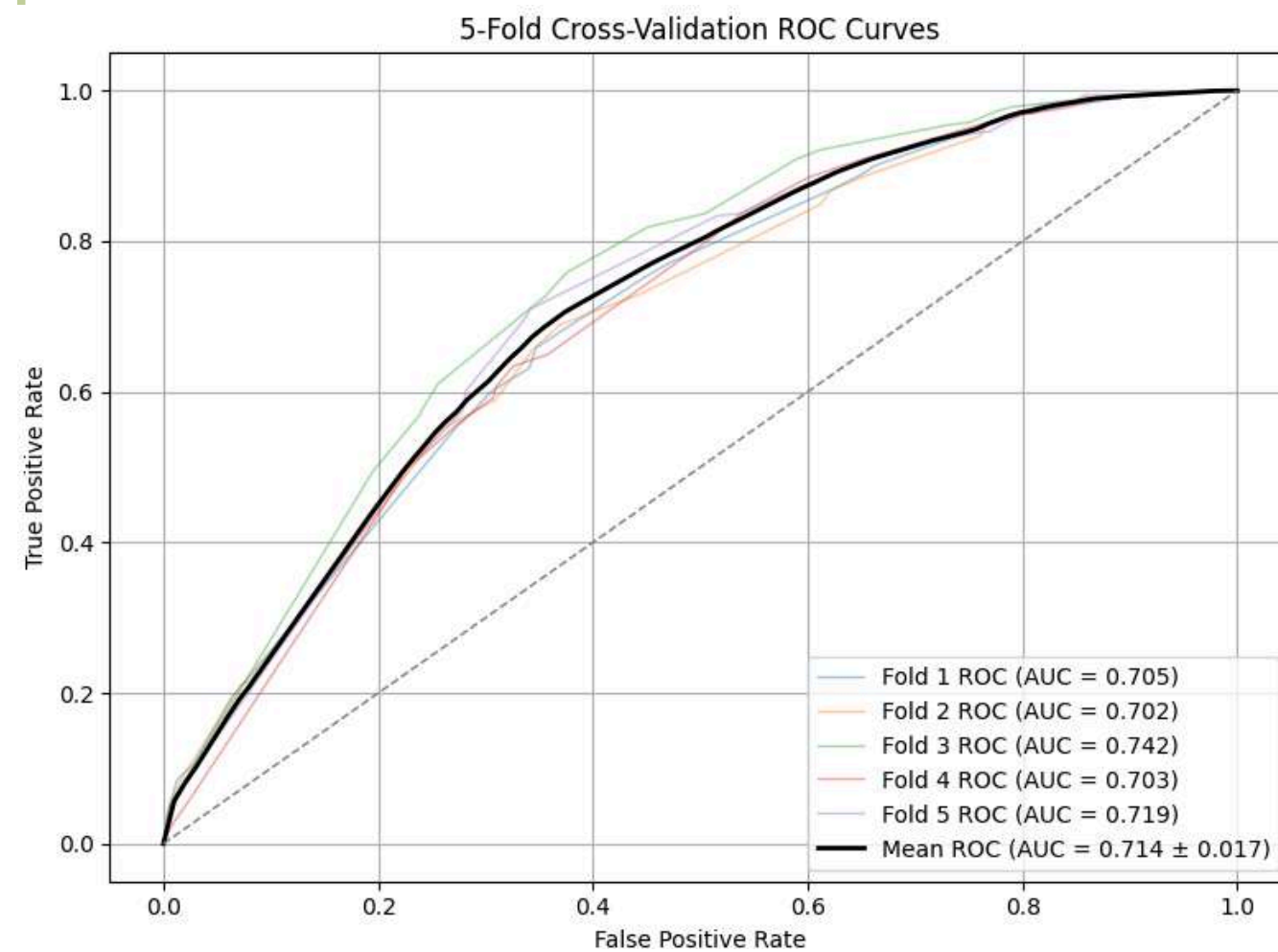
超參數調整

最佳超參數

參數項目	設定值	意義
max_depth	5	• 樹的深度 • 小 → 簡單、偏差大 • 大 / None → 複雜、容易過擬合
min_samples_split	2	• 節點至少要幾筆資料才可以再分 • 大 → 樹比較保守
min_samples_leaf	5	• 葉節點最少樣本數 • 防止只靠極少數樣本做決策
criterion	gini	• gini / entropy • 不同的「純度計算方式」



五折交叉驗證



```
=== 5-fold results (per fold) ===  
Fold 1: pos_rate(train)=0.511 | ACC=0.6559 | F1=0.6616 | AUC=0.7050  
Fold 2: pos_rate(train)=0.511 | ACC=0.6605 | F1=0.6746 | AUC=0.7025  
Fold 3: pos_rate(train)=0.511 | ACC=0.6867 | F1=0.7036 | AUC=0.7417  
Fold 4: pos_rate(train)=0.511 | ACC=0.6538 | F1=0.6522 | AUC=0.7032  
Fold 5: pos_rate(train)=0.511 | ACC=0.6801 | F1=0.6897 | AUC=0.7195  
  
=== 5-fold summary (mean ± std) ===  
ACC: 0.6674 ± 0.0150  
F1 : 0.6763 ± 0.0208  
AUC: 0.7144 ± 0.0168
```

- 平均AUC = 0.7114表示模型具有一定的分辨性
- 標準差Std=0.0168→ 波動程度小

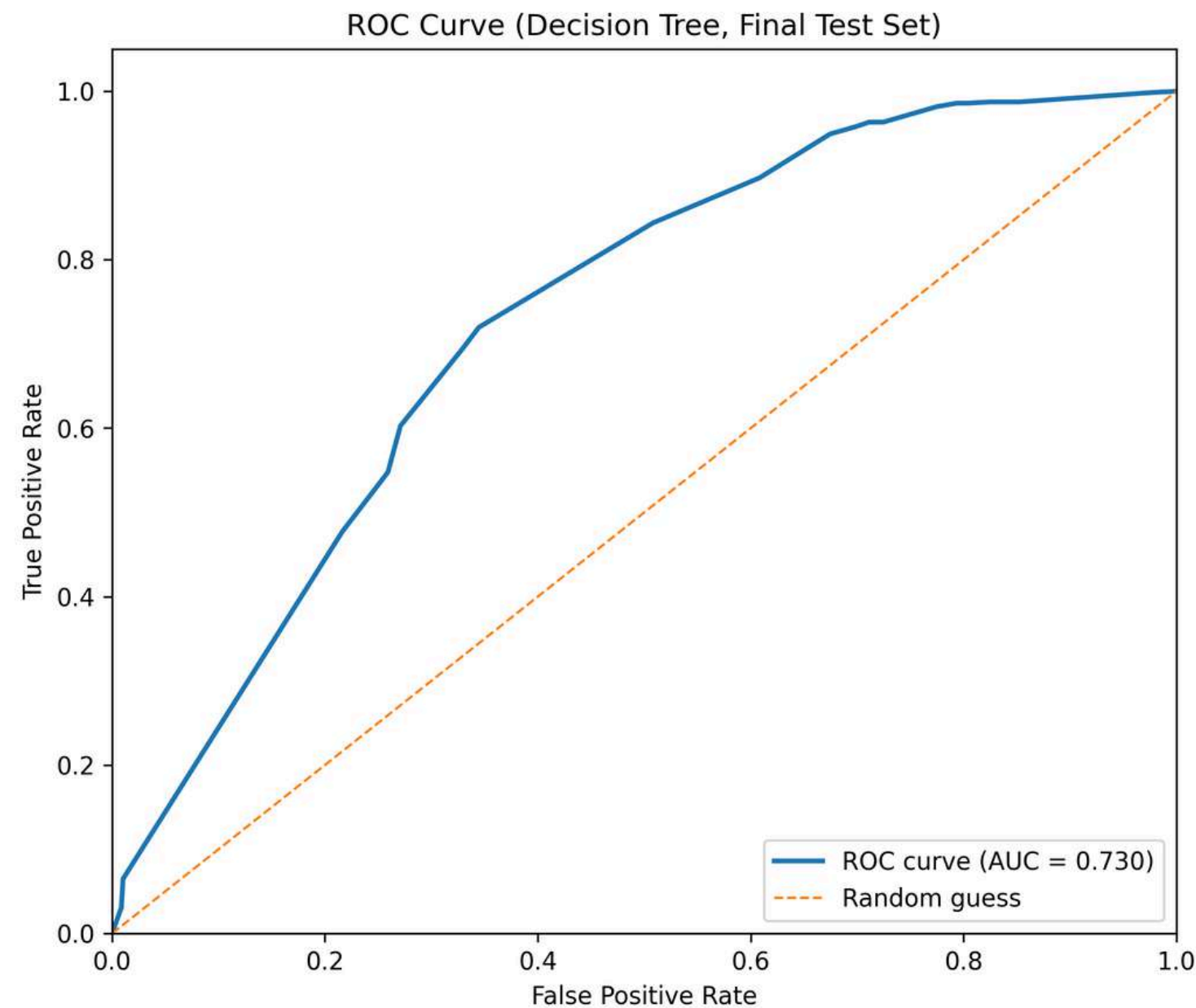
模型訓練

```
model = DecisionTreeClassifier(  
    random_state=RANDOM_STATE,  
    class_weight="balanced",  
    max_depth=5,  
    min_samples_leaf=5,  
    min_samples_split=2,  
    criterion="gini"  
)  
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)  
y_score = model.predict_proba(X_test)[:, 1]  
acc = accuracy_score(y_test, y_pred)  
precision = precision_score(y_test, y_pred, zero_division=0)  
recall = recall_score(y_test, y_pred, zero_division=0)  
f1 = f1_score(y_test, y_pred, zero_division=0)  
auc_roc = roc_auc_score(y_test, y_score)
```

```
=== Final Test Performance (Fixed Hyperparams) ===  
Accuracy : 0.6825  
Precision: 0.6881  
Recall   : 0.6930  
F1       : 0.6905  
AUC      : 0.7296
```

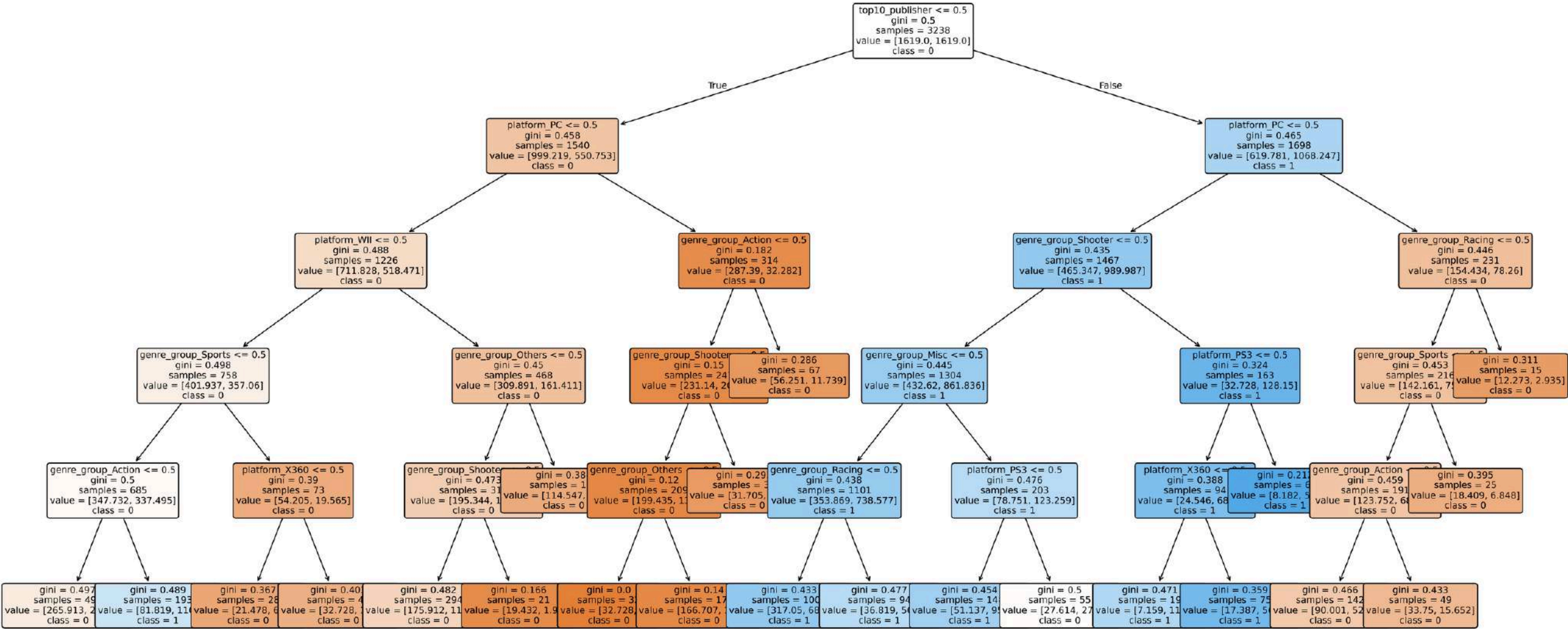
模型訓練



在測試集上，模型 $AUC = 0.7298$ ，說明模型能有效區分正負樣本，且表現略優於五折交叉驗證平均結果 (0.7144)，顯示模型具有穩定且一定的判別能力

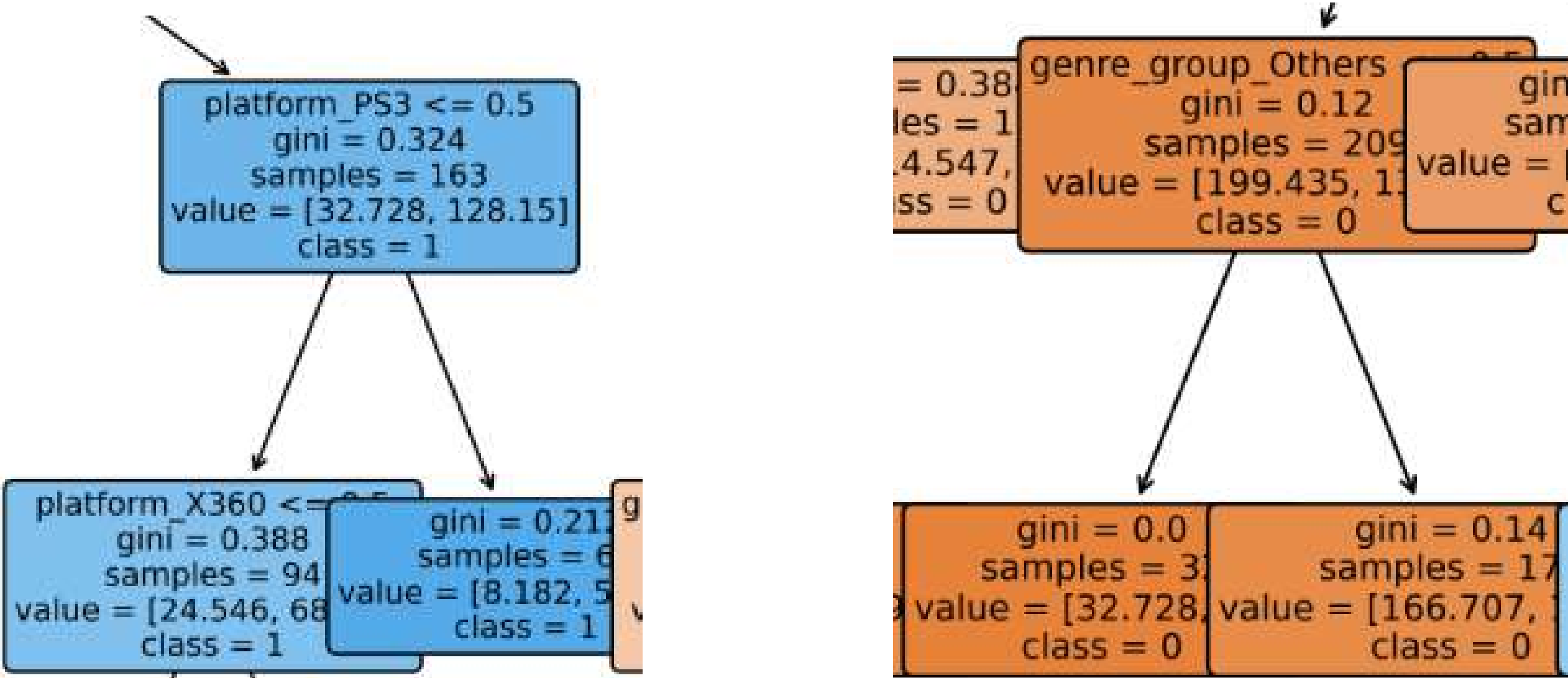
決策樹(訓練完後)

Decision Tree (Fixed Hyperparams) - Top levels



越藍越容易是暢銷遊戲 越黃則越不容易為暢銷遊戲

決策樹(訓練完後)



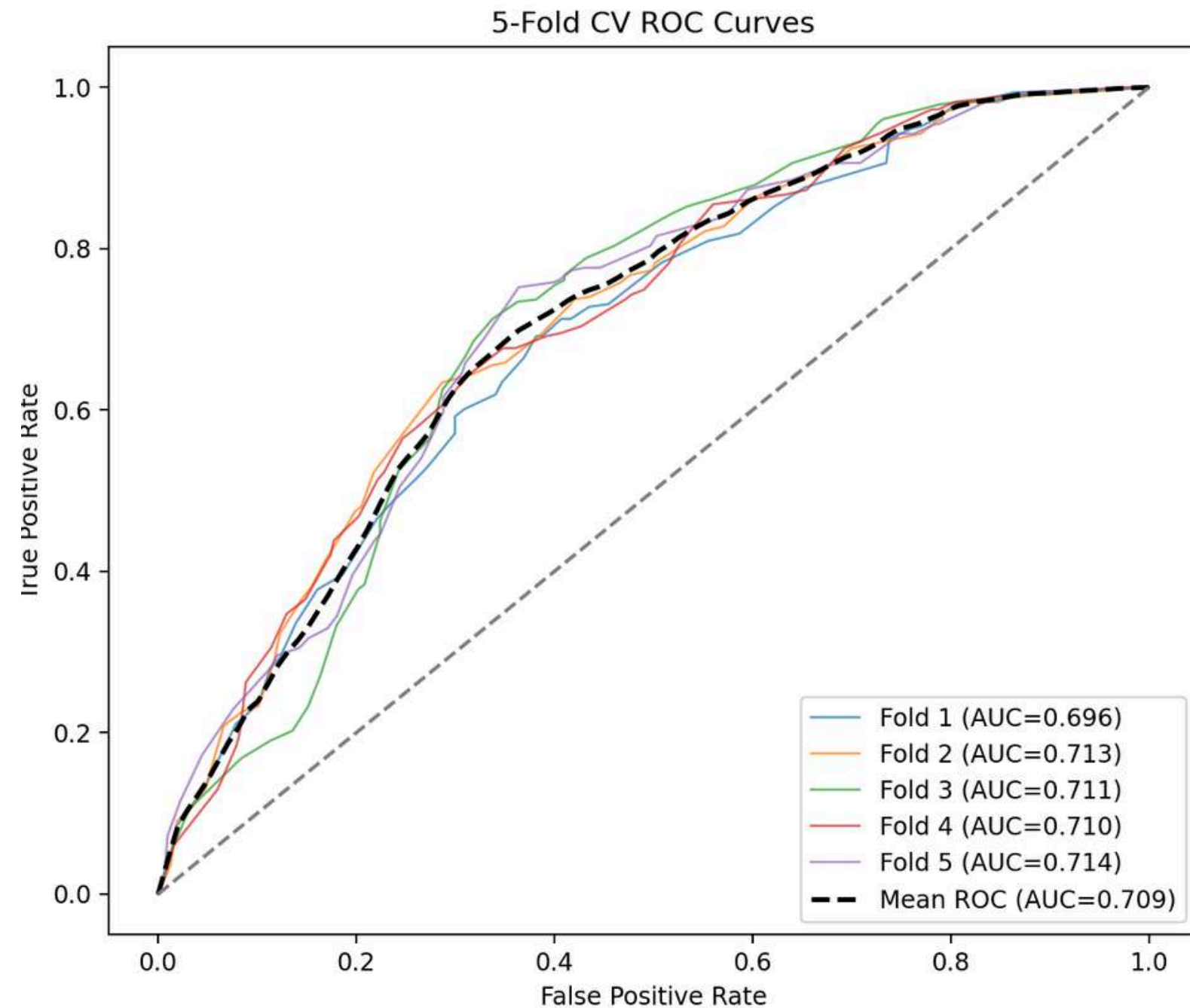
可從前面之節點分析哪些遊戲類型、平台是否是前十大發行商會為HIT（暢銷遊戲）

邏輯斯回歸

五折交叉驗證

```
===== 5-Fold Cross Validation (per fold) =====  
Fold 1: Accuracy=0.6481  
Fold 2: Accuracy=0.6543  
Fold 3: Accuracy=0.6867  
Fold 4: Accuracy=0.6646  
Fold 5: Accuracy=0.6739  
  
平均 CV Accuracy: 0.6655 ± 0.0138  
平均 CV ROC-AUC : 0.7089 ± 0.0067
```

ROC曲線下的面積為0.7089代表
模型有中等的穩定對於區分是否
為暢銷遊戲的能力



五折交叉驗證

```
for train_idx, val_idx in cv.split(X_train, y_train):
    X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

    lr_model.fit(X_tr, y_tr)
    y_proba = lr_model.predict_proba(X_val)[:, 1]
    y_pred = (y_proba >= 0.5).astype(int)

    acc = accuracy_score(y_val, y_pred)
    auc_score = roc_auc_score(y_val, y_proba)

    acc_list.append(acc)
    auc_list.append(auc_score)

    print(f"Fold {fold_idx}: Accuracy={acc:.4f}, ROC-AUC={auc_score:.4f}")
    fold_idx += 1
```



超參數調整

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=5000, random_state=42))
])
```

```
param_grid = [
    {
        "clf__solver": ["liblinear"],
        "clf__penalty": ["l1", "l2"],
        "clf__C": [0.01, 0.1, 1, 10, 100],
        "clf__class_weight": [None, "balanced"]
    },
    {
        "clf__solver": ["lbfgs"],
        "clf__penalty": ["l2"],
        "clf__C": [0.01, 0.1, 1, 10, 100],
        "clf__class_weight": [None, "balanced"]
    }
]
```

```
grid = GridSearchCV(
    pipe,
    param_grid=param_grid,
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1
)
```

```
grid.fit(X_train, y_train)
```

```
print("\n=== GridSearchCV (Train only) ===")
print("Best CV AUC:", grid.best_score_)
print("Best Params:", grid.best_params_)
```

```
=== GridSearchCV (Train only) ===
```

```
Best CV AUC: 0.7100908280657571
```

```
Best Params: {'clf__C': 0.1, 'clf__class_weight': None, 'clf__penalty': 'l1', 'clf__solver': 'liblinear'}
```

C（正則化強度）

控制模型的自由程度

C 小：**限制強**，模型較保守

C 大：**限制弱**，模型較容易**過度擬合**

penalty（正則化方式）

L1：只保留**重要特徵**（部分係數變為 0）

L2：保留**所有特徵**，但影響力較平均

class_weight（類別權重）

None：不調整類別權重

邏輯斯回歸

```
best_lr = grid.best_estimator_

y_pred_tuned = best_lr.predict(X_test)
y_proba_tuned = best_lr.predict_proba(X_test)[:, 1]

acc_tuned = accuracy_score(y_test, y_pred_tuned)
auc_tuned = roc_auc_score(y_test, y_proba_tuned)

print("\n=== Tuned Logistic (Test) ===")
print(f"Accuracy: {acc_tuned:.4f}")
print(f"AUC      : {auc_tuned:.4f}")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_tuned))
print("\nClassification Report:\n", classification_report(y_test, y_pred_tuned, digits=4))
```

=== Tuned Logistic (Test) ===

Accuracy: 0.6765

AUC : 0.7227

Confusion Matrix:

```
[[467 211]
 [238 472]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.6624	0.6888	0.6753	678
1	0.6911	0.6648	0.6777	710
accuracy			0.6765	1388
macro avg	0.6767	0.6768	0.6765	1388
weighted avg	0.6771	0.6765	0.6765	1388

在僅使用平台、類型與發行商資訊的情況下，
Logistic Regression 仍能達到接近 **7 成** 的分類
 正確率，
 顯示這些特徵對暢銷表現具有一定解釋能力。

邏輯斯回歸

```
best_lr = grid.best_estimator_

y_pred_tuned = best_lr.predict(X_test)
y_proba_tuned = best_lr.predict_proba(X_test)[: , 1]

acc_tuned = accuracy_score(y_test, y_pred_tuned)
auc_tuned = roc_auc_score(y_test, y_proba_tuned)

print("\n=== Tuned Logistic (Test) ===")
print(f"Accuracy: {acc_tuned:.4f}")
print(f"AUC      : {auc_tuned:.4f}")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_tuned))
print("\nClassification Report:\n", classification_report(y_test, y_pred_tuned, digits=4))
```

=== Tuned Logistic (Test) ===

Accuracy: 0.6765

AUC : 0.7227

Confusion Matrix:

```
[[467 211]
 [238 472]]
```

Classification Report:

	precision	recall	f1-score	support
0	<u>0.6624</u>	0.6888	0.6753	678
1	0.6911	0.6648	0.6777	710
accuracy			0.6765	1388
macro avg	0.6767	0.6768	0.6765	1388
weighted avg	0.6771	0.6765	0.6765	1388

Recall = 0.6888

☞ 66% 的非暢銷遊戲有被模型成功抓到

Precision $\approx$ 0.6624

☞ 模型判定為非暢銷的遊戲中，約 66% 真的是非暢銷

邏輯斯回歸

```
best_lr = grid.best_estimator_

y_pred_tuned = best_lr.predict(X_test)
y_proba_tuned = best_lr.predict_proba(X_test)[:, 1]

acc_tuned = accuracy_score(y_test, y_pred_tuned)
auc_tuned = roc_auc_score(y_test, y_proba_tuned)

print("\n=== Tuned Logistic (Test) ===")
print(f"Accuracy: {acc_tuned:.4f}")
print(f"AUC      : {auc_tuned:.4f}")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_tuned))
print("\nClassification Report:\n", classification_report(y_test, y_pred_tuned, digits=4))
```

=== Tuned Logistic (Test) ===

Accuracy: 0.6765

AUC : 0.7227

Confusion Matrix:

```
[[467 211]
 [238 472]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.6624	0.6888	0.6753	678
1	<u>0.6911</u>	<u>0.6648</u>	0.6777	710
accuracy			0.6765	1388
macro avg	0.6767	0.6768	0.6765	1388
weighted avg	0.6771	0.6765	0.6765	1388

Recall = 0.6648

☞ 約 71% 的暢銷遊戲有被模型成功抓到

Precision $\approx$ 0.6911

☞ 模型判定為暢銷的遊戲中，約 69% 真的是暢銷

模型對暢銷遊戲具有較低的召回率，
表示其稍微容易漏判真正具有市場潛力的作品。

邏輯斯回歸

```
best_lr = grid.best_estimator_

y_pred_tuned = best_lr.predict(X_test)
y_proba_tuned = best_lr.predict_proba(X_test)[:, 1]

acc_tuned = accuracy_score(y_test, y_pred_tuned)
auc_tuned = roc_auc_score(y_test, y_proba_tuned)

print("\n=== Tuned Logistic (Test) ===")
print(f"Accuracy: {acc_tuned:.4f}")
print(f"AUC      : {auc_tuned:.4f}")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_tuned))
print("\nClassification Report:\n", classification_report(y_test, y_pred_tuned, digits=4))
```

=== Tuned Logistic (Test) ===

Accuracy: 0.6765

AUC : 0.7227

Confusion Matrix:

[[467 211]

[238 472]]

Classification Report:

	precision	recall	f1-score	support
0	0.6624	0.6888	0.6753	678
1	<u>0.6911</u>	<u>0.6648</u>	0.6777	710
accuracy			0.6765	1388
macro avg	0.6767	0.6768	0.6765	1388
weighted avg	0.6771	0.6765	0.6765	1388

macro avg：把 0 和 1 當成一樣重要

weighted avg：依照樣本數加權平均

兩個數字一樣，代表：

類別比例平衡

模型沒有嚴重偏向某一類

邏輯斯回歸

Logistic Regression 在測試集上達到約 67.7% 的準確率，

並對暢銷遊戲（Hit）具有約 66% 的召回率，
顯示平台、遊戲類型與發行商規模對遊戲是否暢銷具
有一定解釋能力。

邏輯斯回歸

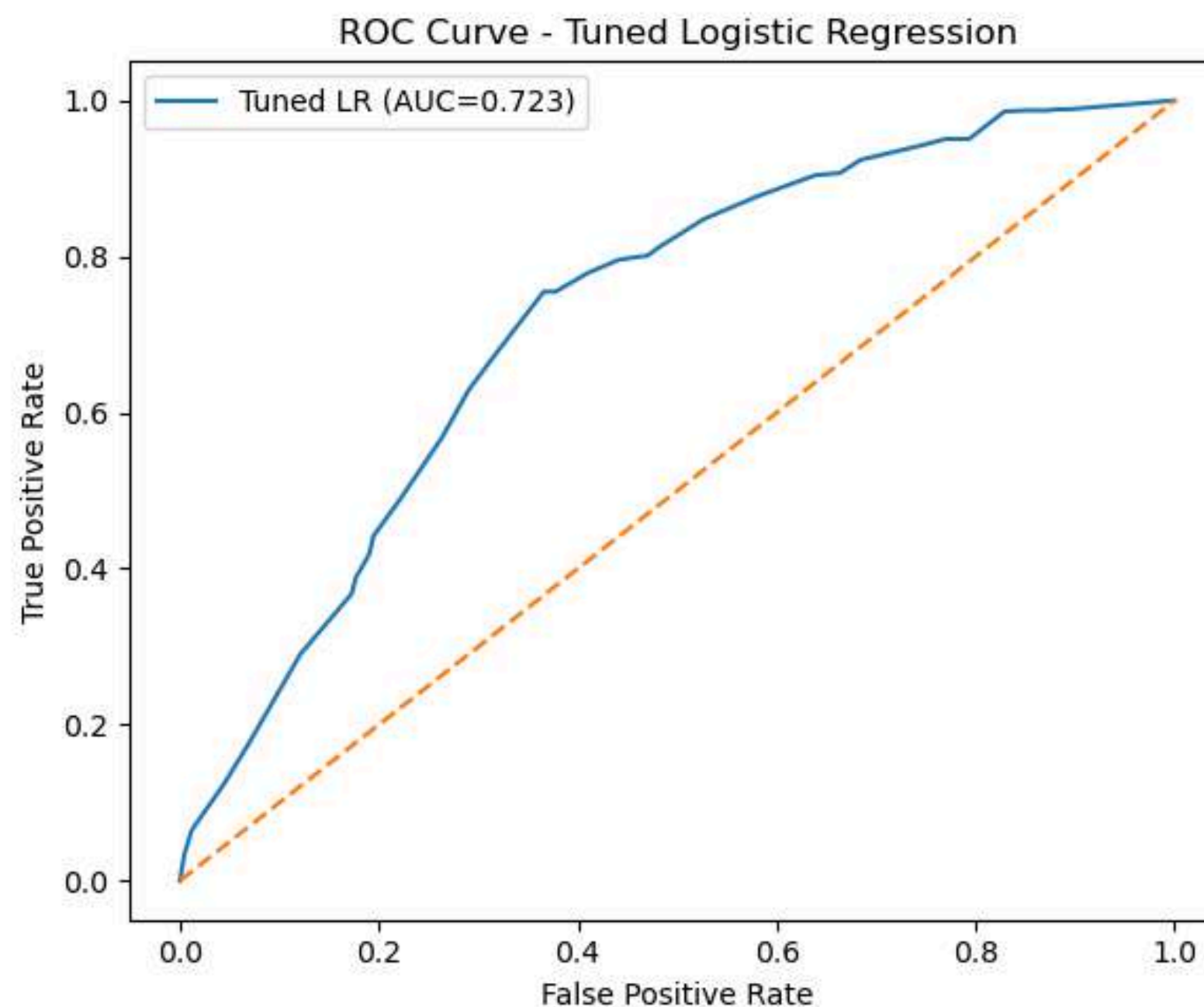
Tuned Accuracy : 0.6765

Tuned AUC : 0.7227

準確率從0.6655->0.6765

ROC曲線下面積從0.7089->0.7227

顯示模型泛化能力良好，未出現過度擬合的問題，也具備中等能夠區分是否為暢銷遊戲的能力





模型精準度比較

	決策樹	邏輯斯回歸
accuracy	0.6825	0.6765
precision	0.6881	0.6911
recall	0.6930	0.6648
f1-score	0.6905	0.6777



由此可見**決策樹**精準度比較高

Q&A1

資料前處理與特徵編碼後，資料實際長什麼樣？還有編碼是如何進行的？

鎖定研究範圍(平台)修改版

先觀察出我們的平台總共有幾種，接著將以品牌的公司進行區分，Sony_PlayStation、Nintendo、Microsoft_Xbox、pc、以及將剩餘的小眾平台合併成Other

平台總數：31	【NINTENDO】	【SONY】	【XBOX】
2600	3DS	PS	X360
3DO	DS	PS2	XB
3DS	GB	PS3	XOne
DC	GBA	PS4	
DS	GC	PSP	
GB	N64	PSV	
GBA	NES		【OTHER】
GC	SNES		2600
GEN	Wii		3DO
GG	WiiU		DC
N64			GEN
NES			GG
NG			NG
PC			PCFX
PCFX			SAT
PS			SCD
PS2			TG16
PS3			WS
PS4			
PSP			
PSV			
SAT			
SCD			
SNES			
TG16			
WS			
Wii			
WiiU			
X360			
XB			
XOne			

【PC】
PC

鎖定研究範圍(遊戲類型)修改版

原始資料包含十多種類型，部分
類型樣本數極少

為避免：

維度過高、模型不穩定

本研究：

依樣本數選出

前五大遊戲類型

其餘類型統一歸為 Others

```
=== Top5 Genres ===  
['Action', 'Sports', 'Misc', 'Role-Playing', 'Shooter']  
genre_group 分布：  
Others          6305  
Action          3253  
Sports          2304  
Misc            1710  
Role-Playing    1471  
Shooter         1282  
Name: genre_group, dtype: int64
```

資料編碼 修改版

平台 (Platform)、遊戲類型 (Genre) 屬於類別型資料
機器學習模型無法直接處理文字類別
需將類別資料轉換為數值形式，才能進行模型訓練

- 採用 **One-Hot Encoding**
- 將每一個平台與遊戲類型轉換為獨立的 0 / 1 特徵欄位
- 使模型能分別學習各類別對「是否成為暢銷遊戲 (Hit)」的影響

=== 平台 One-Hot 欄位順序 ===

```
0 platform_group_NINTENDO
1 platform_group_OTHER
2 platform_group_PC
3 platform_group_SONY
4 platform_group_XBOX
```

=== 類型 One-Hot 欄位順序 ===

```
0 genre_group_Action
1 genre_group_Misc
2 genre_group_Others
3 genre_group_Role-Playing
4 genre_group_Shooter
5 genre_group_Sports
```

=== 平台 類別 -> One-Hot 向量 ===

```
NINTENDO -> 10000
OTHER    -> 01000
PC       -> 00100
SONY     -> 00010
XBOX     -> 00001
```

=== 類型 類別 -> One-Hot 向量 ===

```
Action    -> 100000
Misc      -> 010000
Others    -> 001000
Role-Playing -> 000100
Shooter   -> 000010
Sports    -> 000001
```

經前處理後的資料編碼

特徵	定義	內容	資料型態
Hit(Y)	是否為暢銷遊戲（目標變數）	依 Global_Sales 中位數進行二元分類： 0 = 低於中位數 1 = 高於或等於中位數	二元類別（0 / 1）
platform_group	遊戲所屬平台群組	原始 31 種平台依品牌整併為 5 類： SONY / NINTENDO / XBOX / PC / OTHER	類別型 → One-Hot
genre_group	遊戲類型群組	依樣本數選取前 5 大遊戲類型，其餘合併為 Others	類別型 → One-Hot
top10_publisher	是否為前 10 大暢銷發行商	依 Hit 總數排名前 10 名： 0 = 否，1 = 是	二元類別（0 / 1）

經前處理後的資料編碼

platform_group	One-Hot 向量
NINTENDO	10000
OTHER	01000
PC	00100
SONY	00010
XBOX	00001

經前處理後的資料編碼

genre_group	One-Hot 向量
Action	100000
Misc	010000
Others	001000
Role-Playing	000100
Shooter	000010
Sports	000001

經前處理後的資料編碼

項目	說明
原始資料筆數	16598筆
原始特徵欄位數	11 欄
資料清理方式	移除必要欄位缺失值 (platform / genre / publisher / year / global_sales)
清理後有效資料筆數	16,325 筆
最終輸入特徵 (X)	platform_group、 genre_group、 top10_publisher
目標變數 (Y) Hit (是否為暢銷遊戲)	目標變數 (Y) Hit (是否為暢銷遊戲)

Q&A2

目標變數 Hit 的定義與 0/1 分布是否合理？

HIT 0/1分布

本研究以 Global_Sales 的中位數作為 Hit 的切分標準，
因此 Hit=0 與 Hit=1 的比例接近 50%。
此為刻意設計的結果，
目的是避免類別不平衡，
並提升後續分類模型的穩定性與可解釋性。

=== Hit 數量 ===

0 7998

1 8327

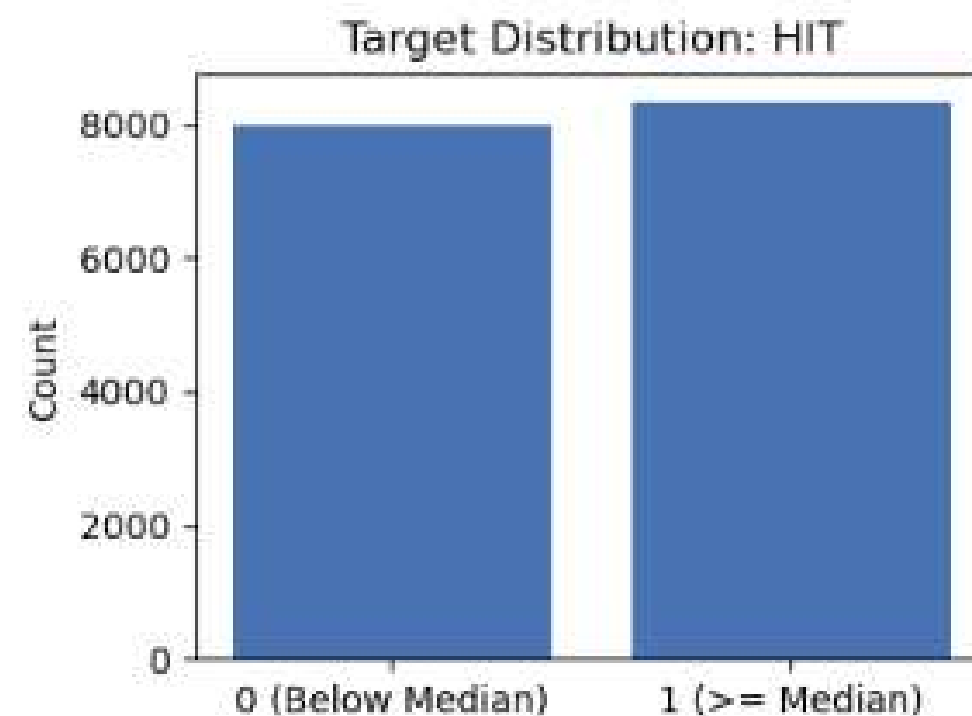
Name: hit, dtype: int64

=== Hit 比例 ===

0 0.489923

1 0.510077

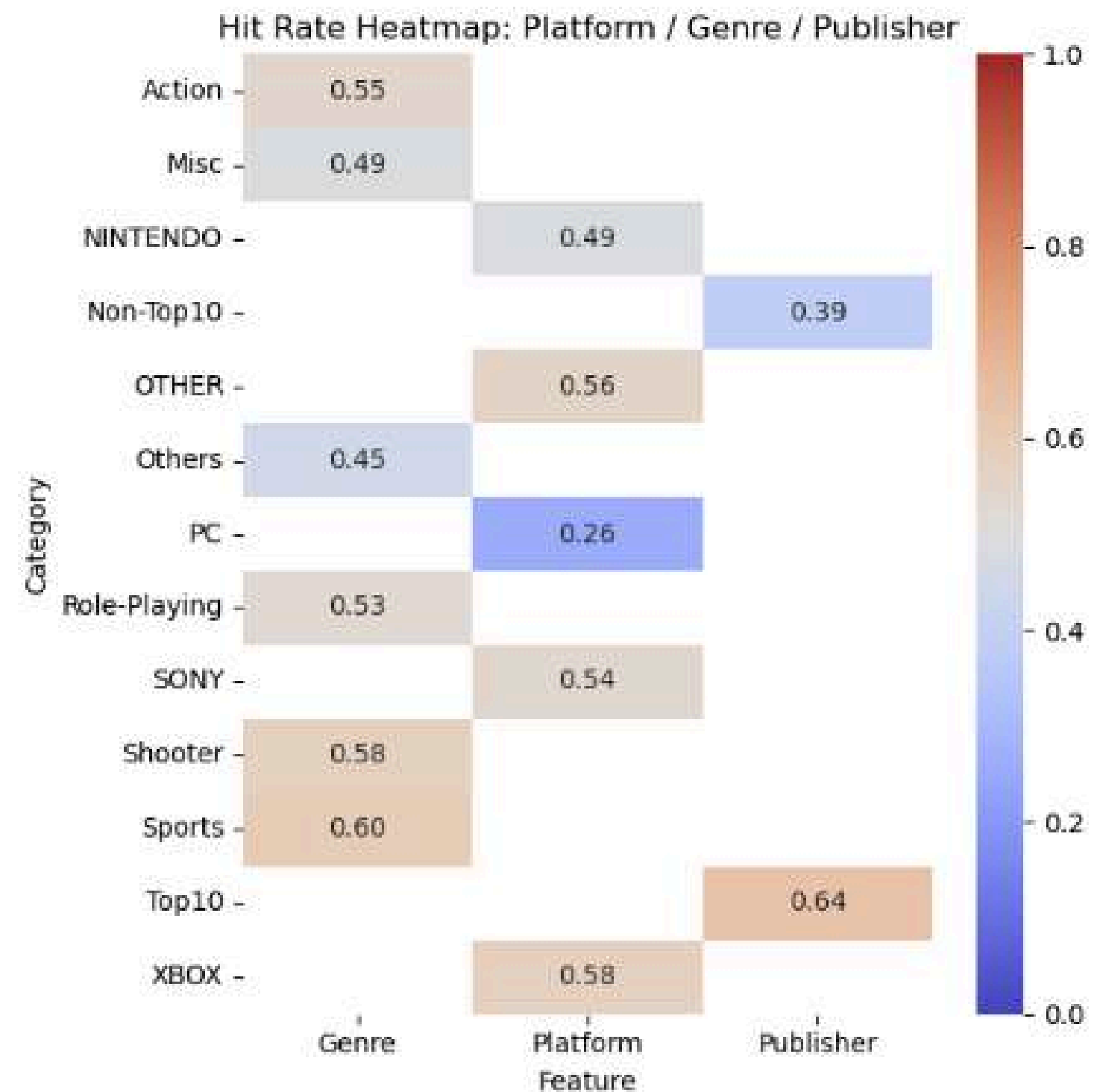
Name: hit, dtype: float64



Q&A3

僅使用 3 個特徵，是否足以解釋 Hit?

三個高層次類別特徵（平台、遊戲類型、發行商）對 Hit 皆呈現一致且具方向性的影響趨勢。整體而言，
 此組特徵對 Hit 具有中等程度的解釋能力，能夠反映部分銷售成功的結構性差異，可作為模型訓練前的重要初步評估依據。



Q4

五折裡面之80趴訓練集如何確認平衡？

A4

```
model_cv = DecisionTreeClassifier(  
    random_state=RANDOM_STATE,  
    class_weight="balanced", 根據類別在資料中的比例自動調整權重，  
    **best_params            減少類別不平衡對模型學習的影響  
)  
skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
```

```
n_pos_fold = int((y_tr == 1).sum())  
n_neg_fold = int((y_tr == 0).sum())  
pos_rate_fold = n_pos_fold / (n_pos_fold + n_neg_fold)
```

計算正樣本在每一折裡占有多少

```
print(f"Fold {fold}: pos_rate(train)={pos_rate_fold:.3f} |
```

我們使用的是**StratifiedFold**(分層交叉驗證)這個方法進行，在分割資料時會依照目標變數的比例進行抽樣，因此每一折中的訓練集都會維持與整體資料集相近的正負類比例，確保資料分布是平衡且一致的

```
Fold 1: pos_rate(train)=0.511  
Fold 2: pos_rate(train)=0.511  
Fold 3: pos_rate(train)=0.511  
Fold 4: pos_rate(train)=0.511  
Fold 5: pos_rate(train)=0.511
```

Q5

最佳超參數尋找過程中是如何設定範圍？

A5

我們決策樹的超參數是依據資料量與模型複雜度控制原則，由小到大逐步擴展，以觀察模型從簡單到複雜時對 AUC 的影響包含：

- **最大深度** (max_depth) [2,3,4,5,6,8,10,None]
- **最小分裂樣本數** (min_samples_split) [2,5,10,20]
- **葉節點最小樣本數** (min_samples_leaf) [1,2,5,10]
- **分裂準則** (criterion) ["gini","entropy"]

參數之範圍源則為：

- **由淺至深**:設定從 2 到 10 以及不限制，是為了比較淺樹、適中深度與完全展開時的效能
- **避免過擬合**:設定不同的 min_samples_split 與 min_samples_leaf，一樣從小到大逐步提升，模擬嚴格與寬鬆分裂條件對過擬合的影響
- **比較分裂準則**:不同的標準對於分類方式會有差別，使用決策樹最常見的兩種標準gini和entropy下去做比較

這些範圍都來自**文獻與 sklearn 預設建議區間**，確保搜尋空間具有**代表性**且可控制計算成本

	max_depth	min_samples_split	min_samples_leaf	criterion	Mean_AUC	Std_AUC
63	5	20	10	gini	0.714358	0.015004
62	5	10	10	gini	0.714358	0.015004
61	5	5	10	gini	0.714358	0.015004
60	5	2	10	gini	0.714358	0.015004
59	5	20	5	gini	0.714358	0.015004
58	5	10	5	gini	0.714358	0.015004
57	5	5	5	gini	0.714358	0.015004
56	5	2	5	gini	0.714358	0.015004
184	5	2	5	entropy	0.714347	0.015014
185	5	5	5	entropy	0.714347	0.015014

總共會跑 $8*4*4*2=256$ 次

Q6

為何結果是看AUC?

AG

AUC衡量的的是模型對正負樣本整體區辨能力（排序能力），不依賴單一分類閾值

例如隨機抽一筆暢銷遊戲和非暢銷遊戲，模型給暢銷遊戲的**預測分數**比非暢銷遊戲高的機率

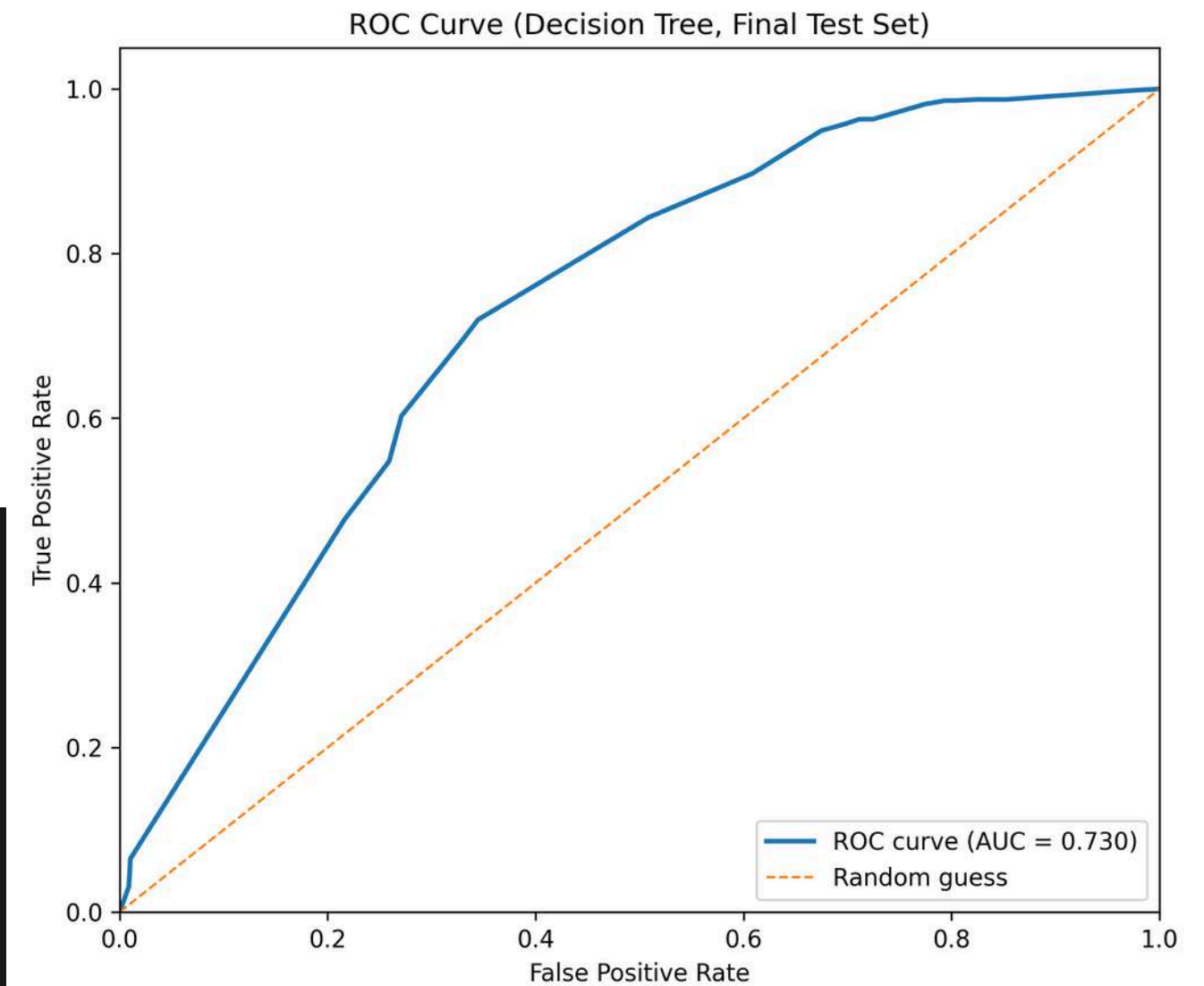
而ACC,Recall,Precision或F1-Score都需要先選一個**閾值**（例如 0.5）才計算，結果會受閾值影響，雖然我們的資料平衡，但假設模型預測暢銷遊戲機率有 0.49, 0.51，模型只用Accuracy,Recall,Precision 或 F1-Score的話，閾值 0.5 可能會**錯過**一些近 0.5 的正類樣本，而AUC可以看到模型整體排序能力，不用設定閾值，也能知道模型好不好

AG

AUC具備之特性:

- 閾值無關
- 尺度 0~1
- 對類別不平衡較穩定
- 反映排序能力而非分類結果

```
=== Final Test Performance (Fixed Hyperparams)
Accuracy : 0.6825
Precision: 0.6881
Recall   : 0.6930
F1       : 0.6905
AUC      : 0.7296
```





Q7

112029038林資竣

怎麼確定73分之後的訓練集以及測試集當中的HIT數量是相同的

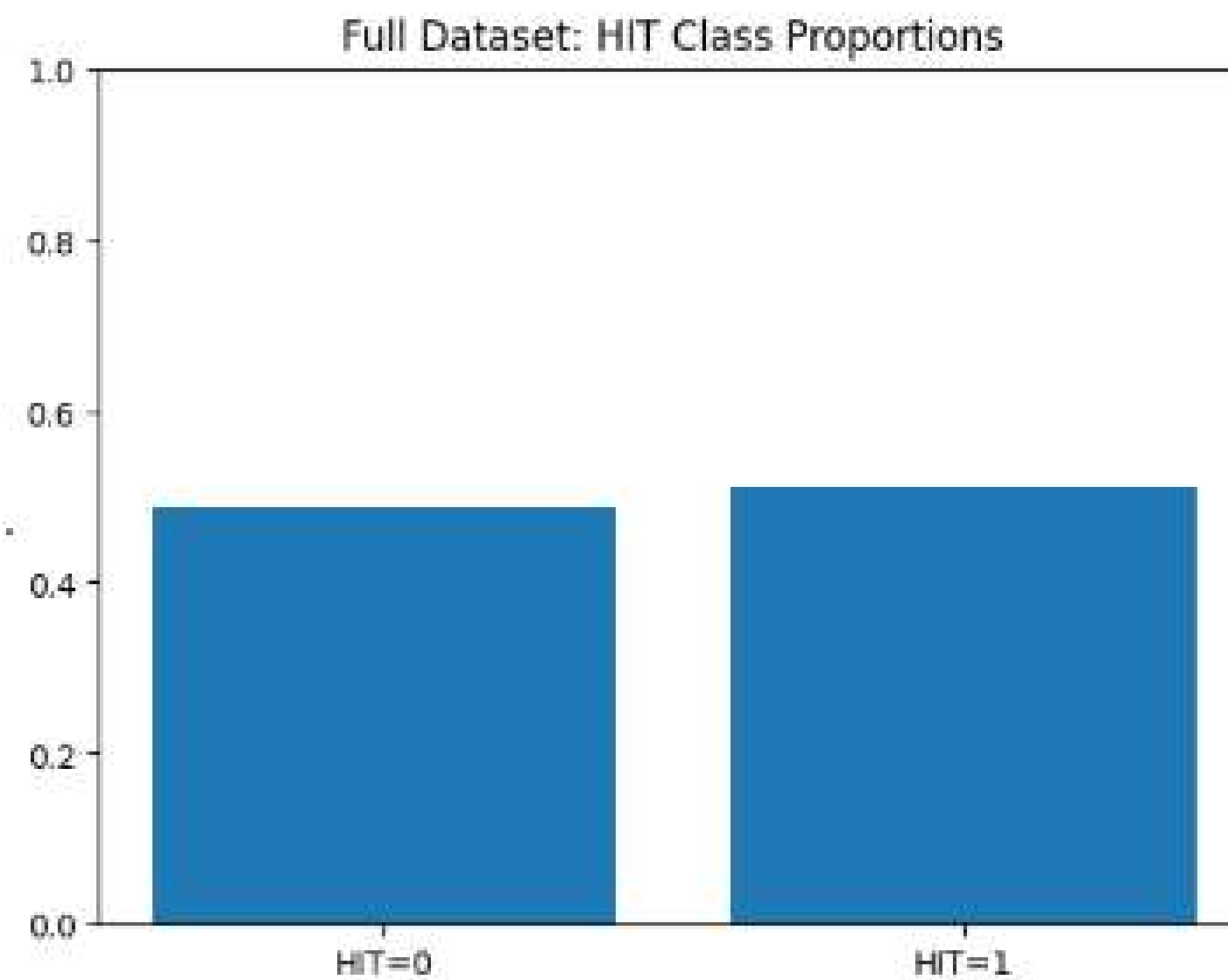
A7

112029038林資竣

首先在一開始的HIT劃分的時候我們是使用中位數來劃分
暢銷遊戲(HIT=1)跟非暢銷遊戲(HIT=0)

前處理過後->

```
=== Full Dataset Summary ===  
Total samples: 4626  
HIT=1 count : 2365  
HIT=0 count : 2261  
HIT rate    : 0.5112
```





A7

112029038林資竣



接著再使用分層抽樣的方式來抽
先抓取HIT=1 切分出訓練集以及測試集
再抓取HIT=0 切分訓練集以及測試集

A7

切分完的資料量

=== Train/Test Summary (Stratified Split) ===

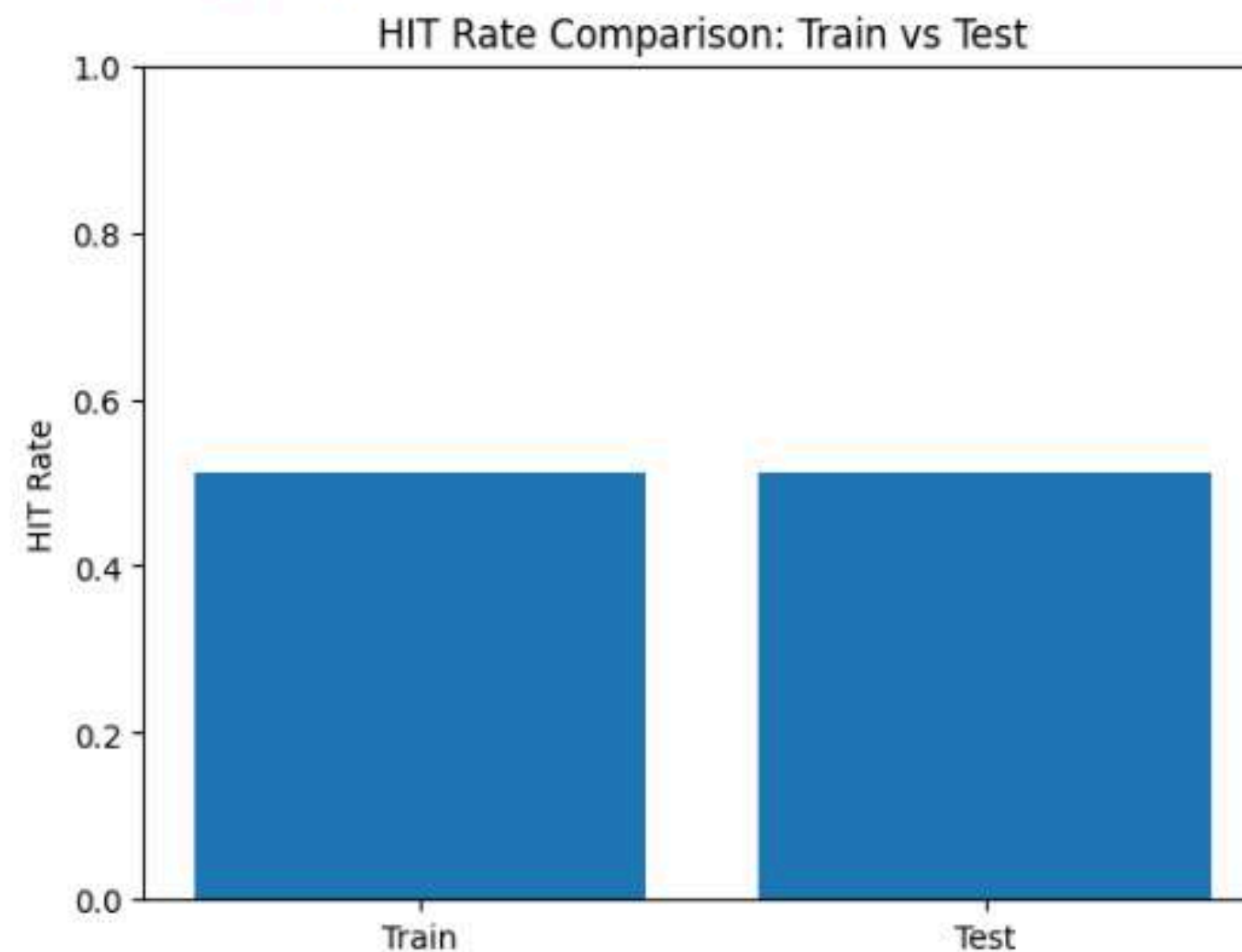
Dataset	Sample Size	HIT=1 Count	HIT=0 Count	HIT Rate
Train	3238	1655	1583	0.511118
Test	1388	710	678	0.511527

	HIT=1	HIT=0
Train	1655	1583
Test	710	678

112029038林資竣

兩者的絕對差值只有0.000409
代表兩者相差不大

HIT Rate 的 絕對差值(Train vs Test): 0.000409



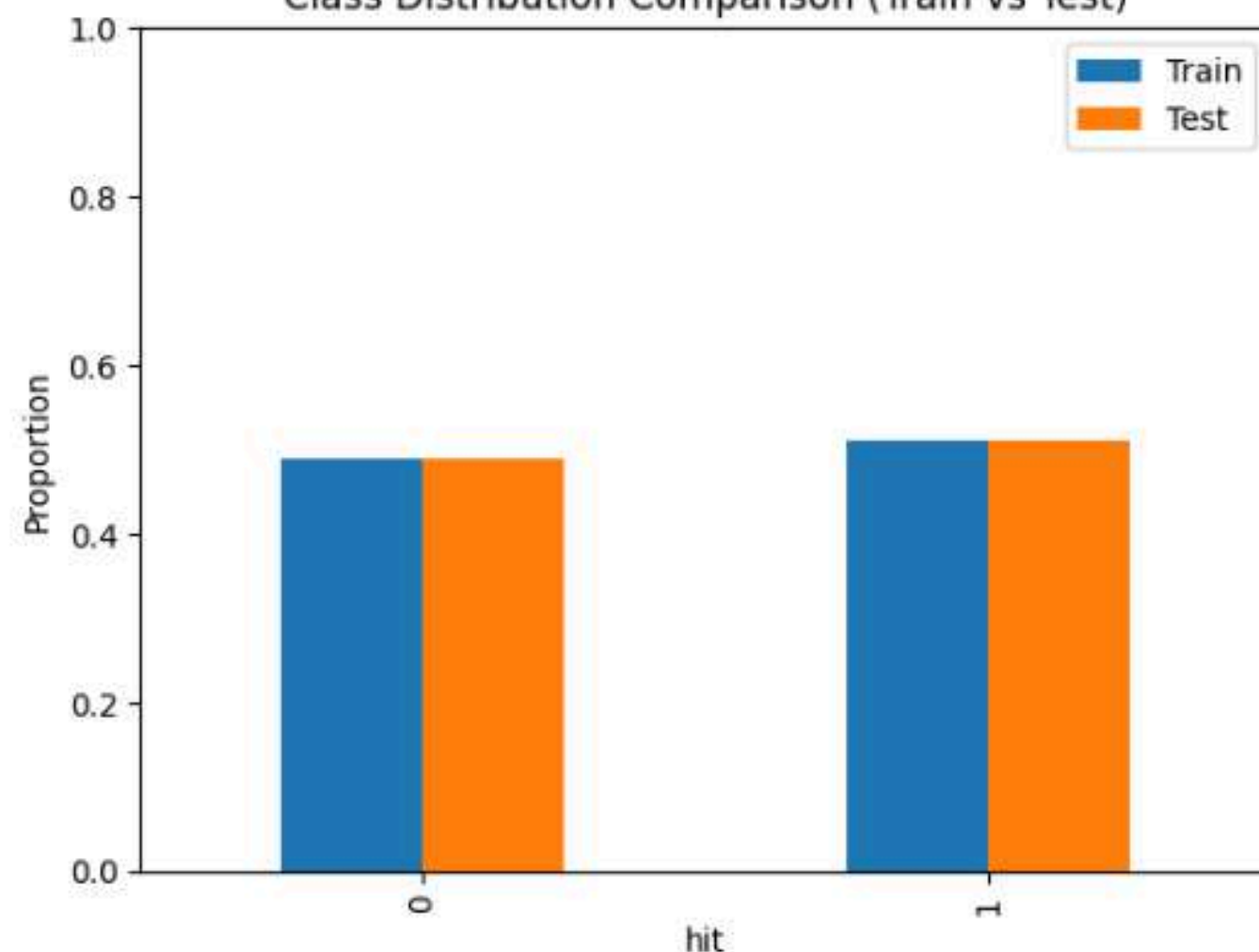
A7

112029038林資竣

=== Class Proportion (0/1) ===

	Train	Test
hit		
0	0.488882	0.488473
1	0.511118	0.511527

Class Distribution Comparison (Train vs Test)



從這些圖中可以知道分層抽樣出來之後
訓練集與測試集的 HIT 比例幾乎一致，表示
兩個資料集在類別分布上保持一致性，不會
因切分造成類別比例偏移



Q8

112029038林資竣

邏輯斯迴歸中的正則化調整的內容是什麼



A8

112029038林資竣



主要針對的部分是調整損失函數 (Loss Function)

C的部分則是調整模型的複雜度

C 大 → 正則化弱 → 模型複雜 → 權重範圍大 (可離原點很遠)

C 小 → 正則化強 → 模型簡單 → 權重被壓近原點

$$\text{正則化強度} = \frac{1}{C}$$



A8

penalty

112029038林資竣

L1 penalty(曼哈頓距離)

$$\text{Manhattan distance} = |x_1| + |x_2| + \dots$$

L2 penalty(歐幾里德距離)

$$\text{Euclidean distance} = \sqrt{x^2 + y^2}$$



A8

penalty

112029038林資竣



L1 penalty(曼哈頓距離)

權重不做調整。

造成稀疏解 (特徵選擇)

以絕對值衡量參數與原點距離，促進
稀疏解

L2 penalty(歐幾里德距離)

權重整體縮小但不為 0

模型平滑、穩定

以平方距離限制參數大小，使模型平
滑



Q9

112029039黃柏誠

AUC數值的五個區間代表是什麼

A9

112029039黃柏誠

在五折交叉驗證時我們模型的平均AUC是0.7089，模型區分正負樣本的能力可接受，在最後測試集下去跑時落在0.7227
所以結果同上

AUC=0.5-0.6	模型區分能力較差
AUC=0.6-0.7	模型區分能力較弱
AUC=0.7-0.8	模型區分能力可接受
AUC=0.8-0.9	模型區分能力佳
AUC>0.9	模型區分能力非常好

文獻Hanley & McNeil (1982)The meaning and use of the area under a receiver operating characteristic (ROC) curve



Q10

112029039黃柏誠

五折交叉驗證中0跟1的比例是多少





A10

```
Fold 1:  
 訓練集 -> 0: 1266 (0.4888), 1: 1324 (0.5112)  
 驗證集 -> 0: 317 (0.4892), 1: 331 (0.5108)  
Fold 1: Accuracy=0.6481  
Fold 2:  
 訓練集 -> 0: 1266 (0.4888), 1: 1324 (0.5112)  
 驗證集 -> 0: 317 (0.4892), 1: 331 (0.5108)  
Fold 2: Accuracy=0.6543  
Fold 3:  
 訓練集 -> 0: 1266 (0.4888), 1: 1324 (0.5112)  
 驗證集 -> 0: 317 (0.4892), 1: 331 (0.5108)  
Fold 3: Accuracy=0.6867  
Fold 4:  
 訓練集 -> 0: 1267 (0.4890), 1: 1324 (0.5110)  
 驗證集 -> 0: 316 (0.4884), 1: 331 (0.5116)  
Fold 4: Accuracy=0.6646  
Fold 5:  
 訓練集 -> 0: 1267 (0.4890), 1: 1324 (0.5110)  
 驗證集 -> 0: 316 (0.4884), 1: 331 (0.5116)  
Fold 5: Accuracy=0.6739
```

因為我們的訓練集的筆數是偶數，
所以會出現某幾折的數量會差一筆
，但還是可以看出我們的資料分布
是蠻平均的，不需另做類別平衡

訓練集的0跟1的比例

112029039黃柏誠

	0	1
Fold1	1266(49%)	1324(51%)
Fold2	1266(49%)	1324(51%)
Fold3	1266(49%)	1324(51%)
Fold4	1267(49%)	1324(51%)
Fold5	1267(49%)	1324(51%)





A10

112029039黃柏誠

驗證集的0跟1的比例

```
Fold 1:
  訓練集 -> 0: 1266 (0.4888), 1: 1324 (0.5112)
  驗證集 -> 0: 317 (0.4892), 1: 331 (0.5108)
Fold 1: Accuracy=0.6481
Fold 2:
  訓練集 -> 0: 1266 (0.4888), 1: 1324 (0.5112)
  驗證集 -> 0: 317 (0.4892), 1: 331 (0.5108)
Fold 2: Accuracy=0.6543
Fold 3:
  訓練集 -> 0: 1266 (0.4888), 1: 1324 (0.5112)
  驗證集 -> 0: 317 (0.4892), 1: 331 (0.5108)
Fold 3: Accuracy=0.6867
Fold 4:
  訓練集 -> 0: 1267 (0.4890), 1: 1324 (0.5110)
  驗證集 -> 0: 316 (0.4884), 1: 331 (0.5116)
Fold 4: Accuracy=0.6646
Fold 5:
  訓練集 -> 0: 1267 (0.4890), 1: 1324 (0.5110)
  驗證集 -> 0: 316 (0.4884), 1: 331 (0.5116)
Fold 5: Accuracy=0.6739
```

	0	1
Fold1	317(49%)	331(51%)
Fold2	317(49%)	331(51%)
Fold3	317(49%)	331(51%)
Fold4	317(49%)	331(51%)
Fold5	317(49%)	331(51%)





Q11

112029038林資竣

為何最後做出來的模型兩者的數據都大約只有在7成

	決策樹	邏輯斯回歸
accuracy	0.6825	0.6765
precision	0.6881	0.6911
recall	0.6930	0.6648
f1-score	0.6905	0.6777



A11

112029038林資竣

因為我們當初在做資料前處理的時候，我們將各地區的資料銷售量給刪除了。原先的資料欄位有11個不同的變數，在我們的考量下我們最後只剩下3筆特徵去訓練模型，因此就會導致模型在評估過程中能判斷的數據過少，容易造成誤判。

參考文獻

<https://medium.com/python-資料視覺化/python-商業數據分析之可視化繪圖-第19講-熱力圖-seaborn-heatmap-cf1b17d7964e>

<https://medium.com/python-資料視覺化/python-商業數據分析之可視化繪圖-第8講-散點圖-scatter-chart-d693c18a40a6>

<https://ithelp.ithome.com.tw/m/articles/10322270>

分組討論

112012067秦晨恩



資竣 下午11:38
這邊約一下下次討論的時間吧 一樣是周六晚上討論嗎? 還是有人這周不行 (已編輯)

晨恩 下午11:39
我週六晚上可以!

柏誠 下午11:39
我可以

柏堯  下午11:40
我可以

雲翊 下午11:41
我可能周日才可
sor

資竣 下午11:42
那就週日8.嗎?

柏誠 下午11:42
ok



資竣 下午3:55
大家這周甚麼時候有空
我們訂一天晚上來討論嗎?
我禮拜三 五 六 日 都可以

晨恩 下午3:58
我五六日都可以

柏誠 下午3:58
我都可以

雲翊 下午3:58
我也都可以!

資竣 下午4:02
@柏堯 你呢

柏堯  下午4:09
都可以👍

資竣 下午4:10
那我們就六日晚上8.後討論!

柏誠 下午4:12
ok

雲翊 下午4:12
好勒

分組討論

112012067秦晨恩

語音頻道 ▾

🔊 一般 3:35:00

設定頻道狀態 ✎

- 晨恩
- 柏堯 illit
- 柏誠
- 資竣
- 雲翊

語音頻道 ▾

🔊 一般 3:29:23

- 晨恩
- 柏堯 illit
- 柏誠
- 資竣
- 雲翊

🔊 一般 4:46:57

設定頻道狀態 ✎

- 資竣 直播
- 晨恩
- 柏堯 illit
- 柏誠
- 雲翊

語音頻道 ▾

🔊 一般 4:38:46

設定頻道狀態 ✎

- 柏堯 illit 直播
- 資竣 直播
- 晨恩
- 柏誠
- 雲翊

工作分配

112029038 林資竣	做簡報 背景 研究目的 資料描述 資料型態 資料前處理 邏輯斯回歸含五折分析 Q7-8&A Q11&A
112029039 黃柏誠	做簡報 預期結果 邏輯斯回歸
112029023 游雲翊	做簡報 問題思考 Q&A 決策樹
112029051 黃柏堯	做簡報 背景 研究目的 資料描述 資料型態 資料前處理 資料視覺化 QA
112012067 秦晨恩	做簡報 問題思考 模型比較



THANK YOU